



EÖTVÖS LORÁND UNIVERSITY

FACULTY OF INFORMATICS

DEPT. OF PROGRAMMING LANGUAGES AND COMPILERS

**Intrinsic proof of
completeness of Beth models
for intuitionistic first-order logic**

Supervisor:

Szumi Xie

PhD student

Author:

Korpa Péter Zsolt

Computer Science MSc

Budapest, 2026

DIPLOMAMUNKA TÉMABEJELENTŐ

Hallgató adatai:

Név: Korpa Péter Zsolt

Neptun kód: QKFN4M

Képzési adatok:

Szak: programtervező informatikus, mesterképzés (MA/MSc)

Tagozat : Nappali

Belső témavezetővel rendelkezem

Témavezető neve: Xie Zongpu

munkahelyének neve, tanszéke: ELTE IK, Programozási nyelvek és Fordítóprogramok Tanszék

munkahelyének címe: 1117, Budapest, Pázmány Péter sétány 1/C.

beosztás és iskolai végzettsége: PhD hallgató

Specializáció : Szoftvertechnológia

A diplomamunka címe: Intrinsic proof of Completeness of Beth models for intuitionistic First-order logic

A diplomamunka témája:

(A témavezetővel konzultálva adja meg 1/2 - 1 oldal terjedelemben diplomamunka témájának leírását)

First-order logic is one of the most studied formal systems and is the backbone of modern foundations of mathematics. Completeness is an important property of models of logical theories. In this thesis we formalize the Completeness proof of Beth models for intuitionistic First-order logic. Our formalization is the first intrinsic one, written in the Agda programming language. We utilize techniques from meta-theory of type theory, such as, generalized algebraic theories for describing First-order logic, and presheaf/sheaf semantics for describing Beth models. We also explore other models, and the double negation translation. Our formalization gives a better understanding of the connections between different models of intuitionistic First-order logic such as Tarski models, Kripke models and Beth models. We also believe this different way to view Beth/Sheaf style semantics could prove valuable for further research in meta-theory of type theory.

Budapest, 2025. 10. 14.

Contents

1	Introduction	3
2	Related Work	5
2.1	Categorical logic	5
2.2	Other formalisations	5
3	Second-order generalised algebraic theories	6
3.1	GATs	6
3.2	SOGATs	9
4	Logic as a SOGAT	12
4.1	Propositional logic as a SOGAT	12
4.2	First-order logic as a SOGAT	14
5	Translating FOL to a GAT	16
5.1	SOGAT to GAT translation	16
5.2	Applying the translation to $\mathbf{Fol}_{SOGAT}$	18
5.3	Splitting the context	19
6	Syntax as a definable quotient	22
6.1	Syntax as a IIT	22
6.2	Iterator	26
7	Completeness for Propositional logic	30
7.1	Completeness	30
7.2	Kripke semantics	31
7.3	Soundness and Completeness for Kripke models	33
8	Completeness Theorems for First-order logic	37
8.1	Beth semantics	37

8.2 Soundness and completeness for Beth models	44
9 Double-negation translation	49
10 Further research	55
11 Conclusion	56
Acknowledgements	57
A Appendix	58
A.1 Metatheory	58
A.2 Category theory	59
Bibliography	60

Chapter 1

Introduction

First-order logic is a historically and practically important theory which has been studied for a long time. There are various kinds of descriptions and formulations but most of them are extrinsic, that means that they are described in an untyped way [1]. These presentations can cause lots of preservation lemmas [2] which are usually not expanded on, but in formalisation they can cause lots of tedious work. In contrast a typed (intrinsic) presentation can avoid this, by indexing. That is instead of using relations, it enforces constraints via indexing.

The goal of this thesis is to formalise first-order logic (FOL) in an intrinsically typed manner as a SOGAT [3]. Second-order generalised algebraic theories are generalised algebraic theories with a notion of variables and binders baked in. This style allows for a simpler presentation that is also easy to work in, and are nicer to define [4, 5]. We use a translation given by Kaposi and Xie [6] to translate our definition to a generalised algebraic theory, which we can use to give models of our theory. We show that the notion of model we get is the expected notion of first-order logic by defining the Beth model of first-order logic, and also proving completeness for Beth models. We also show how the syntax of this theory can be given as a definable quotient, meaning only using inductive data types. Our contributions :

- Formalised proof of completeness of Beth models for intuitionistic first-order logic.
- Showing that the syntax of our notion of FOL is a definable quotient and proving the initiality of the syntax.
- We formalise the double negation translation of FOL.

These results are formalised in the proof assistant Agda and are available online () . When a definition has corresponding code we put a little Agda logo next to it () . This formalisation also includes the formalisation of Kripke and Beth models for propositional and first-order logic, and Beth completeness for the propositional case as well.

In chapter 3 and chapter 4 we describe SOGATs and first-order logic as a SOGAT. In chapter 5 we describe and apply the SOGAT to GAT translation to logic, and give the syntax in chapter 6. We then describe propositional Kripke models in chapter 7 and prove a completeness result for a fragment of propositional logic. In chapter 8 we do this for Beth models and in chapter 9 we give the double negation translation. In Appendix A we explain notations and give a few categorical definitions.

Chapter 2

Related Work

2.1 Categorical logic

Categorical logic is about applying category theory to the study of mathematical logic. In categorical logic, logical theories are presented as categories with some additional properties like monoidal categories or coherent categories. Models of theories are then functors from these categories into some other categories, for example set models go into the category of sets. A proof of completeness of Beth models is given in Julia Morin’s masters thesis [7]. Categorical logic is usually done in a classical setting so some of the arguments, for example in the proof of completeness of Beth models doesn’t work constructively. More about categorical logic can be found in Mihály Makkai and Gonzalo E. Reyes’s book on Categorical logic [8].

2.2 Other formalisations

First-order logic has been formalised in many forms, extensional and intension. Andreas Halkjær From gives a formalisation of a natural deduction style proof system in Isabell and proves completeness for a Hilbert style system for first-order logic [9]. There are also formalisations in other proof assistants such as the Formalise Formal Logic project [10]. These formalisation are usually for classical logic or work in a classical metatheory.

Chapter 3

Second-order generalised algebraic theories

Second-order generalised algebraic theories (SOGATs) were introduced by Taichi Uemura in his PhD thesis [3] on generalisations of type theories. SOGATs can be seen as generalisations of Cartmell's generalized algebraic theories (GATs) [11]. They extend GATs with second order operations which allows them to easily model languages with binders. SOGATs and GATs enjoy many nice properties related to their models, which is why we chose to present first-order logic as a SOGAT. In this chapter we present GATs and SOGATs by examples found in Kaposi and Xie's paper about SOGATs [6], we describe GAT morphisms and the category of models of a GAT [12].

3.1 GATs

On a high level generalised algebraic theories consists of the following components:

- Sorts
- Operations
- Equations

Sorts are the "kinds" of elements the theory deals with. They can be indexed by previous sorts, in addition they can even depend on anything defined earlier.

Operations are n-ary functions, the types of these operations can also depend on previous things. Equations are between elements of sorts. We show a few examples of GATs.

Monoids Monoids are sets with a nullary and binary operation such that the binary operation is associative and the nullary operation is the identity.

$$\begin{aligned} \mathbf{M} & : \textit{Sort} \\ \mathbf{e} & : \mathbf{M} \\ - \circ - & : \mathbf{M} \rightarrow \mathbf{M} \rightarrow \mathbf{M} \\ \mathbf{e\text{-}left\text{-}identity} & : \mathbf{e} \circ \mathbf{a} = \mathbf{a} \\ \mathbf{e\text{-}right\text{-}identity} & : \mathbf{a} \circ \mathbf{e} = \mathbf{a} \\ \mathbf{comp\text{-}is\text{-}assoc} & : (\mathbf{a} \circ \mathbf{b}) \circ \mathbf{c} = \mathbf{a} \circ (\mathbf{b} \circ \mathbf{c}) \end{aligned}$$

We have a single Sort that we call $\mathbf{M}$. $\mathbf{e} : \mathbf{M}$ says that $\mathbf{e}$ is in the Sort $\mathbf{M}$. $- \circ -$ is the binary operation on $\mathbf{M}$, and $\mathbf{e\text{-}left\text{-}identity}$, $\mathbf{e\text{-}right\text{-}identity}$, $\mathbf{comp\text{-}is\text{-}assoc}$ express the equalities.

Graphs with a single reflexive point

$$\begin{aligned} \mathbf{Node} & : \textit{Sort} \\ \mathbf{Edge} & : \mathbf{Node} \rightarrow \mathbf{Node} \rightarrow \textit{Sort} \\ \mathbf{a} & : \mathbf{Node} \\ \mathbf{refl\text{-}a} & : \mathbf{Edge} \mathbf{a} \mathbf{a} \end{aligned}$$

We have two sorts, $\mathbf{Node}$ and $\mathbf{Edge}$, which are the points of the Graph and the edges between them. The sort $\mathbf{Edge}$ is given as a indexed sort over the two endpoints of the edge. There are two nullary operations, called constans, $\mathbf{a}$ and $\mathbf{refl\text{-}a}$ which corresponds to a point and a reflexive edge on that point. The operation $\mathbf{refl\text{-}a}$ depends on the previously defined constant $\mathbf{a}$.

Preorders Preorders are partial orders where we don't require the that the ordering is anti-reflexive. That is they have a binary predicate which is reflexive and transitive. The equality $\mathbf{p\text{-}prop}$ states that there is at most one way to be "less then equal".

$$\begin{aligned}
 P & & : \text{Sort} \\
 - \leq - & & : P \rightarrow P \rightarrow \text{Sort} \\
 \text{p-prop} & & \forall(a, b : P). \forall(p, q : a \leq b). p = q \\
 \text{reflexivity} & & : n \leq n \\
 \text{transitivity} & & : n \leq m \rightarrow m \leq l \rightarrow n \leq l
 \end{aligned}$$

A model of a theory is a way to interpret the Sort as sets, and operations as functions on said sets, such that the equalities hold. An example is natural numbers forming a model of the theory of monoids.

Example 1 (Natural numbers as a Monoid).

$$\begin{aligned}
 M & & := \mathbb{N} \\
 e & & := 0 \\
 n \circ m & & := n + m
 \end{aligned}$$

There is a natural way to define homomorphism between models of a given GAT $\mathcal{G}$. We give an informal definition.

Definition 1 (Model homomorphism of GATs). For a given GAT $\mathcal{G}$ a homomorphism between two of its models M and N has the following components

- For each sort of $\mathcal{G}$ a function between the corresponding sets in M and N .
- For each operation of $\mathcal{G}$ an equality stating that the functions commute with the operations.

With this notion of homomorphism we can give a high level definition of the category of models of a GAT.

Definition 2 (Category of models). For a GAT $\mathcal{G}$ its category of models has the following components

- Objects are models of $\mathcal{G}$
- Morphisms are model homomorphism
- There is an identity model homomorphism for any model

- Composition of homomorphism is given component by component of the functions F_S .

With a precise definition of GATs one can check that the required equations of a category hold. Furthermore there is an initial object of the category of models, we call this object the syntax. The initiality of the syntax means that there is a unique model homomorphism from it into any other model, we call this morphisms the iterator. A more detailed account and definition of GATs can be found in John Cartmell's PhD thesis [11].

3.2 SOGATs

SOGATs extend GATs with second order operations. A second order operation is one which takes a function as a parameter. That is, while an operation like

$$- \circ - : M \rightarrow M \rightarrow M$$

is first order, an operation like

$$f : (M \rightarrow M) \rightarrow M$$

is second order. On a high level SOGATs give us the power to work easily with variables and binders, which are natural parts of languages.

In formal languages, and some mathematical notations such as the integral/summation notation, we use variables and binders.

$$\sum_{i=0}^{10} f(i) = \sum_{i=5}^{15} f(i-5) \tag{3.1}$$

$$\int_{-\infty}^{\infty} e^{-x^2} dx = \sqrt{\pi} \tag{3.2}$$

In Equation 3.1, i is a variable, and the summation sign is a binder. We intuitively know that the i in the left hand side is not the same as the i on the right, because they are bound by different binders. In Equation 3.2 the variable x is bound by the integral.

Usually the semantics of variables are hard to pin down, there are many equivalence definitions but subtle differences can result in unforeseen consequences.

SOGATs allow us to easily introduce binders into our theory with second order operations. It automatically gives a nice semantics for them [6].

Natural examples of SOGATs are languages rather than everyday mathematical objects. We give a simple example of a SOGAT, the language of Church's lambda calculus [13].

Definition 3 (Lambda calculus).

$$\begin{aligned} \mathsf{Tm} & : \mathit{Sort} \\ - \cdot - & : \mathsf{Tm} \rightarrow \mathsf{Tm} \rightarrow \mathsf{Tm} \\ \mathsf{lam} & : (\mathsf{Tm} \rightarrow \mathsf{Tm}) \rightarrow \mathsf{Tm} \end{aligned}$$

We have one sort, the sort of terms Tm , and two operations. The first one, $- \cdot -$, is application and the other one is lambda, or abstraction, usually written with an actual lambda (λ) but we will call it lam . lam is the second order operator of our theory, it takes an actual (meta-theoretic) function and produces an element of Tm . We say lam binds a variable

Example 2 (Constant term). $\mathsf{lam}(x \mapsto \mathsf{lam}(y \mapsto x))$

Example 3 (S combinator term). $\mathsf{lam}(f \mapsto \mathsf{lam}(g \mapsto \mathsf{lam}(x \mapsto f \cdot x \cdot (g \cdot x))))$

Example 4 (Y combinator term). $\mathsf{lam}(f \mapsto \mathsf{lam}(x \mapsto f \cdot (x \cdot x)) \cdot \mathsf{lam}(x \mapsto f \cdot (x \cdot x)))$

We use the notation $a \mapsto b$ for a meta level function which maps a to b . One can extend this language with a notion of types to arrive at the theory of simple types or simply typed lambda calculus.

Definition 4 (Simply typed lambda calculus).

$$\begin{aligned} \mathsf{Ty} & : \mathit{Sort} \\ \mathsf{Tm} & : \mathsf{Ty} \rightarrow \mathit{Sort} \\ \iota & : \mathsf{Ty} \\ - \Rightarrow - & : \mathsf{Ty} \rightarrow \mathsf{Ty} \rightarrow \mathsf{Ty} \\ \mathsf{lam} & : \forall (A B : \mathsf{Ty}). (\mathsf{Tm} A \rightarrow \mathsf{Tm} B) \rightarrow \mathsf{Tm} (A \Rightarrow B) \\ - \cdot - & : \forall (A B : \mathsf{Ty}). \mathsf{Tm} (A \Rightarrow B) \rightarrow (\mathsf{Tm} A \rightarrow \mathsf{Tm} B) \\ \Rightarrow_\beta & : \forall f a. (\mathsf{lam} f) \cdot a = f a \\ \Rightarrow_\eta & : \forall f. \mathsf{lam}(z \mapsto f \cdot z) = f \end{aligned}$$

We use the notation $f\ a$ for meta level type application, that is $f(a)$. Simply typed lambda calculus has a sort of types and a sort of terms indexed over types. Types can be formed by a trivial type ι and by function types which we denote by the binary infix operator $- \Rightarrow -$. All terms are still a function (**lam**) or a function applied to another term $(-\cdot-)$, but being typed disallows us from forming certain terms, like the Y combinator in example Example 4. The operation $-\cdot-$ takes a term of type $(A \Rightarrow B)$ and a term of type A and produces a term of type B . **lam** now gives back a typed term. In this presentation, with a little bracketing we can see the trick in action:

$$\begin{aligned} \text{lam} & : (\text{Tm } A \rightarrow \text{Tm } B) \rightarrow \text{Tm } (A \Rightarrow B) \\ -\cdot- & : \text{Tm } (A \Rightarrow B) \rightarrow (\text{Tm } A \rightarrow \text{Tm } B) \end{aligned}$$

One can say that **lam** and $-\cdot-$ state that the function space in our theory (terms of type $A \Rightarrow B$) is isomorphic to the functions from our meta theory:

$$\text{Tm } (A \Rightarrow B) \cong \text{Tm } A \rightarrow \text{Tm } B$$

The $\Rightarrow_\beta$ and $\Rightarrow_\eta$ rules say that composing **lam** with $-\cdot-$ and vice-versa is the identity. This presentation has historically been used in embedded language development going by the name higher-order abstract syntax (HOAS) [14, 2]. The categorical semantics and more explanation of SOGATs can be found in Taichi Uemura's PhD thesis [3].

Chapter 4

Logic as a SOGAT

In this chapter we present propositional and first-order logic as a SOGAT as shown by Kaposi and Xie [6].

4.1 Propositional logic as a SOGAT

For simplicity we first introduce propositional logic then we extend it with the first order quantifiers. Propositional logic deals with propositions and their proofs. We form propositions by gluing together atomic formulas. Our theory of propositional logic is parameterized by a set of atomic formulas which we will call *Atom*. We can combine atomic formulas with conjunction $\wedge$, disjunction $\vee$, implication $\supset$ to form formulas. We also have an "always true" $\top$ and "always false" formula $\perp$. Traditional presentations of logic separate the proof calculus from the definition of logic but we combine these two. We have a natural deduction style proof system. We present propositional logic with inference rules like

$$\frac{\text{assumption}}{\text{conclusion}} \text{ rule}$$

This can be read as "if **assumption** then given **rule** we can infer **conclusion**". These rules can easily be translated to SOGAT operations. The above would become

$$\text{rule} : \text{assumption} \rightarrow \text{conclusion}$$

.

We can also have multiple assumptions. If we have $\Gamma \vdash A$ in the assumptions that translates to a second order operations like

$$\frac{\Gamma \vdash A}{B} \text{ rule}_2$$

would translate to

$$\text{rule}_2 : (\Gamma \rightarrow A) \rightarrow B$$

so we can think of $\vdash$ as the second order arrow. Inference rules are widely used and also convenient in that they enforces second orderness, as we are not allowed to write $(A \vdash B) \vdash C$ in the assumptions.

Definition 5 ($\text{Prop}_{\text{SOGAT}}$).

$$\begin{array}{c}
 \frac{}{\text{For} : \text{Sort}} \quad \frac{A : \text{For}}{\text{Pf } A : \text{Sort}} \quad \frac{A : \text{For} \quad B : \text{For}}{A \wedge B : \text{For}} \quad \frac{A : \text{For} \quad B : \text{For}}{A \vee B : \text{For}} \\
 \\
 \frac{A : \text{For} \quad B : \text{For}}{A \supset B : \text{For}} \quad \frac{}{\perp : \text{For}} \quad \frac{}{\top : \text{For}} \quad \frac{A : \text{Atom}}{\text{atom } A : \text{For}} \\
 \\
 \frac{l : \text{Pf } A \quad k : \text{Pf } A}{k = l} \text{ (irrev)} \quad \frac{\text{Pf } A \quad \text{Pf } B}{\text{Pf } (A \wedge B)} \text{ (\wedge intro)} \quad \frac{\text{Pf } (A \wedge B)}{\text{Pf } A} \text{ (\wedge elim}_1\text{)} \\
 \\
 \frac{\text{Pf } (A \wedge B)}{\text{Pf } B} \text{ (\wedge elim}_2\text{)} \quad \frac{\text{Pf } A}{\text{Pf } (A \vee B)} \text{ (\vee intro}_1\text{)} \quad \frac{\text{Pf } B}{\text{Pf } (A \vee B)} \text{ (\vee intro}_2\text{)} \\
 \\
 \frac{\text{Pf } A \vdash \text{Pf } C \quad \text{Pf } B \vdash \text{Pf } C}{\text{Pf } C} \text{ (Velim)} \quad \frac{\text{Pf } (A \vee B)}{\text{Pf } (A \supset B)} \text{ (\vee intro)} \\
 \\
 \frac{\text{Pf } (A \supset B)}{\text{Pf } A \vdash \text{Pf } B} \text{ (\supset elim)} \quad \frac{}{\text{Pf } \top} \text{ (\top intro)} \quad \frac{\text{Pf } \perp}{\text{Pf } C} \text{ (\perp elim)}
 \end{array}$$

The first two inference rules define the sorts. We use **atom** to express atomic formulas. The **intro** and **elim** rules are the natural deduction style introduction and elimination rules. For example, the rule $\wedge \text{intro}$ says that given a proof of a formula A and a proof of a formula B , we can prove $A \wedge B$. The rule $\supset \text{intro}$ and $\supset \text{elim}$ are the same ones we saw in Definition Definition 4 for **lam** and $- \cdot -$. The rule **irrev** states that elements of $\text{Pf } A$ are all equal, that is the formula A can have at most one proof. We can define negation of a formula in terms of $\supset$ and $\perp$.

$$\neg A := A \supset \perp$$

We also define *modus ponens* from $\supset$ elim.

$$\frac{\text{Pf } A \supset B \quad \text{Pf } A}{\text{Pf } B} (\$)$$

$\text{Prop}_{\text{SOGAT}}$ is called intuitionistic propositional logic, to make it classical we need to add one more rule.

$$\frac{\text{Pf } \neg \neg A}{\text{Pf } A} (\text{DNE})$$

We will call the version of $\text{Prop}_{\text{SOGAT}}$ with DNE $\text{Prop}_{\text{SOGAT}}^C$ and without it $\text{Prop}_{\text{SOGAT}}^I$.

4.2 First-order logic as a SOGAT

First-order logic introduces the idea of universal and existential quantification. They are used to express statements such as "For every natural number n , n is either even or odd". First order formulas can have terms and term variables in them, which can then be quantified with the quantifiers. The inverse view is that these quantifiers act as binders and introduce new variables which can be used inside terms. Term are a parameter of our theory like *Atom* was for propositional logic, but here we have functions and relations. That is we have for every n natural number (the arity) a set of function symbols, and a set of relation symbols.

$$\begin{aligned} funar &: \mathbb{N} \rightarrow \text{Set} \\ relar &: \mathbb{N} \rightarrow \text{Set} \end{aligned}$$

We then introduce a new sort of terms **Tm**. Formulas can depend on terms and there is a way to form formulas from relation symbols. We define first-order logic as a SOGAT by extending the definition of $\text{Prop}_{\text{SOGAT}}^I$

Definition 6 ($\text{Fol}_{\text{SOGAT}}^I$).

$$\begin{array}{c}
 \frac{}{\text{Tm} : \text{Sort}} \quad \frac{t : \text{Tm} \vdash A : \text{For}}{\forall t.A : \text{For}} \quad \frac{t : \text{Tm} \vdash A : \text{For}}{\exists t.A : \text{For}} \quad \frac{t t' : \text{Tm}}{\text{Eq } t t' : \text{For}} \\
 \\
 \frac{t : \text{Tm} \vdash \text{Pf } A}{\text{Pf } (\forall x.A)} (\forall \text{intro}) \quad \frac{\text{Pf } (\forall x.A) \quad t : \text{Tm}}{\text{Pf } (A \ t)} (\forall \text{elim}) \\
 \\
 \frac{t : \text{Tm} \quad \text{Pf } (A \ t)}{\text{Pf } (\exists x.A)} (\exists \text{intro}) \quad \frac{t : \text{Tm}, \text{Pf } (A \ t) \vdash \text{Pf } C \quad \text{Pf } (\exists x.A)}{\text{Pf } C} (\exists \text{elim}) \\
 \\
 \frac{t : \text{Tm}}{\text{Pf } (\text{Eq } t t)} (\text{refl}) \quad \frac{t t' : \text{Tm} \quad \text{Pf } (K \ t) \quad \text{Pf } (\text{Eq } t t')}{\text{Pf } (K \ t')} (\text{subst}) \\
 \\
 \dots (\text{components of } \text{Prop}_{\text{SOGAT}}^I)
 \end{array}$$

The introduction rules for $\forall$ says that if we can prove the predicate A for a generic term t , then we can conclude that its true for all t . The elimination rule states that if we know a predicate A holds for all t , and we have a specific term x then we can conclude that A holds for x . The **intro** and **elim** rules for $\exists$ a little bit more complicated. The introduction rules says that if there is a $t : \text{Tm}$ such that $A \ t$ hold then we can conclude $\exists t.A$. The elimination rules says to use a proof of $\exists t.A$ to prove some C , we assume that there is a $t : \text{Tm}$ such that $A \ t$ and from this we can prove C . **Eq** $t t'$ is used to state the equality of two terms, its introduction rule is **refl** which says that any term t is equal to itself. Its elimination rule is **subst** which gives a way to prove a predicate mentioning t by substituting in an equal term t' into it.

Chapter 5

Translating FOL to a GAT

In this chapter we sketch a general procedure introduced by Kaposi and Xie [6], that turns SOGATs into GATs. Then we apply this translation to our definition of first-order logic to obtain the GAT that we use to prove the completeness results in chapter 7 and chapter 8. While working in SOGATs is very ergonomic, proving theorems about theories described as SOGATs requires complex machinery [15].

For this reason we need to translate SOGATs into GATs, as GATs admit nice properties like the previously showed category of models.

5.1 SOGAT to GAT translation

To translate the SOGAT $\mathcal{S}$, into a GAT $\mathcal{G}$ we do the following steps:

For every second-order operation

$$f : (P_{1,1} \rightarrow P_{2,1} \rightarrow \dots \rightarrow P_{n,1} \rightarrow K_1) \rightarrow (P_{1,2} \rightarrow \dots \rightarrow P_{n,2} \rightarrow K_2) \rightarrow \dots \rightarrow L$$

the sort of any elements in positions $P_{n,m}$ is marked with a "+".

$\mathcal{G}$ will have sorts **Con** and **Sub** $\Gamma \Delta$. These are the sort of contexts, which stores variables, and substitutions between contexts.

$$\text{Con} : \text{Sort}$$

$$\text{Sub} : \text{Con} \rightarrow \text{Con} \rightarrow \text{Sort}$$

Con and **Sub** $\Gamma \Delta$ form a category, meaning $\mathcal{G}$ has operations id and $- \circ -$, and equations that state id is unital and $- \circ -$ is associative.

$$\begin{aligned} \text{id} & : \text{Sub } \Gamma \Gamma \\ - \circ - & : \text{Sub } \Delta \Theta \rightarrow \text{Sub } \Gamma \Delta \rightarrow \text{Sub } \Gamma \Theta \\ \text{idl} & : \text{id} \circ \gamma = \gamma \\ \text{idr} & : \gamma \circ \text{id} = \gamma \\ \text{assoc} & : (\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta) \end{aligned}$$

A terminal object in the category **Con** written $\diamond$, called the empty context and an operation ϵ with an equation witnessing the terminality of $\diamond$.

$$\begin{aligned} \diamond & : \text{Con} \\ \epsilon & : \text{Sub } \Gamma \diamond \\ \epsilon\eta & : \gamma = \eta \end{aligned}$$

For each Sort, for example $A_{\mathcal{S}}^+$ in $\mathcal{S} : \text{Sort}$ a $A_{\mathcal{G}} : \Gamma \rightarrow \text{Sort}$, which is a sort indexed by a context Γ and a substitution operation $-[-]_{A_{\mathcal{G}}^+}$ and equations that relate id , $- \circ -$ and the substitution operation on $A_{\mathcal{G}}^+$.

$$\begin{aligned} A_{\mathcal{G}}^+ & : \Gamma \rightarrow \text{Sort} \\ - [A_{\mathcal{G}}^+]^- & : A_{\mathcal{G}}^+ \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow A_{\mathcal{G}}^+ \Delta \\ - [A_{\mathcal{G}}^+]^{\text{id}} & : t[A_{\mathcal{G}}^+]^{\text{id}} = t \\ - [A_{\mathcal{G}}^+]^{\circ} & : t[A_{\mathcal{G}}^+]^{\text{f}\circ\text{g}} = (t[A_{\mathcal{G}}^+]^{\text{f}})[A_{\mathcal{G}}^+]^{\text{g}} \end{aligned}$$

For each marked sort, a context extension operations $\triangleright_{A_{\mathcal{G}}^+}$ for every $A_{\mathcal{G}}^+$.

$$\text{Sub } \Delta (\Gamma \triangleright_{A_{\mathcal{G}}^+}) \cong \text{Sub } \Delta \Gamma \times A_{\mathcal{G}}^+ \Gamma$$

For each operation $f_{\mathcal{S}}$ in $\mathcal{S}$, an operation $f_{\mathcal{G}}$ in $\mathcal{G}$. If the operation was a second-order operation then its function parameters get expressed via context extension.

The category of **Con** and **Sub** model variables and binders. **Con** is a list storing the local variables in scope, the different context extensions are the list extension operation denoting what kind of variable has been introduced. Binders are operations

which make use of this context extension. Variables are essentially De Bruijn indices, that can be selected from a context via $\mathbf{q}$ and $-\mathbf{[p]}$. For example the second De Bruijn index is

$$(\mathbf{q}[\mathbf{p}])[\mathbf{p}]$$

5.2 Applying the translation to $\mathbf{Fol}_{SOGAT}$

In this section we apply the above translation to our definition of first-order logic defined in Definition 6. FOL has two marked sorts, these are the sort of terms $\mathbf{Tm}$ and the sort of proofs $\mathbf{Pf}$.

That means that first-order logic has two notions of variables, these are the term variables which are bound by the quantifiers ($\forall$ and $\exists$), and proof variables which are bound in the introduction, elimination rules of the proof system. The translation generates a category of context and substitutions with a terminal object ($\diamond$) called the empty context. Because we have two notion of variables we have two ways to extend our context with them, these will be the term context extension ($-\triangleright_t$) and the proof context extension ($-\triangleright_p$). The term context extension is like the successor function for natural numbers, it just remembers that a new term variables is in context, while the proof context extension is typed, it also has to remember the "type" of the proof, that is the formula for which this variable is a proof for. Each sort from the original theory get indexed by this context, that is $\mathbf{For}$, $\mathbf{Tm}$ and $\mathbf{Pf}$ all get indexed. They also receive a substitution operation with equations, in categorical terms they become presheafs over the category $\mathbf{Con}$.

All the components of the translated GAT are:

Definition 7 ($\text{Fol}_{\text{GAT}} (\mathcal{U})$).

Con	$: \text{Sort}$	$\multimap_t : \text{Con} \rightarrow \text{Con}$	
Sub	$: \text{Con} \rightarrow \text{Con} \rightarrow \text{Sort}$	$\multimap_{,t} : \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub } \Delta (\Gamma \multimap_t)$	
id	$: \text{Sub } \Gamma \Gamma$	$\mathbf{p}_t : \text{Sub } (\Gamma \multimap_t) \Gamma$	
$\multimap \multimap$	$: \text{Sub } \Delta \Gamma \rightarrow \text{Sub } \Theta \Delta \rightarrow \text{Sub } \Theta \Gamma$	$\mathbf{q}_t : \text{Tm } (\Gamma \multimap_t)$	
idl	$: \text{id} \circ \gamma = \gamma$	$\multimap_t \beta_1 : (\mathbf{p}_t \circ (\gamma_{,t} t)) = \gamma$	
idr	$: \gamma \circ \text{id} = \gamma$	$\multimap_t \beta_2 : (\mathbf{q}_t [\gamma_{,t} t]^t) = t$	
assoc	$: (\gamma \circ \delta) \circ \theta = \gamma \circ (\delta \circ \theta)$	$\multimap_t \eta : ((\mathbf{p}_t \circ \gamma)_{,t} (\mathbf{q}_t [\gamma]^t)) = \gamma$	
$\diamond$	$: \text{Con}$	$\text{Pf} : (\Gamma : \text{Con}) \rightarrow \text{For } \Gamma \rightarrow \text{Sort}$	
ϵ	$: \text{Sub } \Gamma \diamond$	$\text{pfisprop} : (k \text{ l} : \text{Pf } \Delta K) \rightarrow k = l$	
$\diamond \eta$	$: (\sigma : \text{Sub } \Gamma \diamond) \rightarrow \sigma = \epsilon$	$\multimap_p : (\Gamma : \text{Con}) \rightarrow \text{For } \Gamma \rightarrow \text{Con}$	
For	$: \text{Con} \rightarrow \text{Sort}$	$\multimap_{,p} : (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Pf } \Delta (K[\gamma]^F) \rightarrow \text{Sub } \Delta (\Gamma \multimap_p K)$	
$\multimap [-]^F$	$: \text{For } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{For } \Delta$	$\mathbf{p}_p : \text{Sub } (\Gamma \multimap_p K) \Gamma$	
$[\text{id}]^F$	$: A [\text{id}]^F = A$	$\mathbf{q}_p : \text{Pf } (\Gamma \multimap_p K) (K[\mathbf{p}_p]^F)$	
$[\circ]^F$	$: A [\gamma \circ \delta]^F = (A [\gamma]^F) [\delta]^F$	$\multimap_p \beta_1 : p_p \circ (\gamma_{,p} k) = \gamma$	
Tm	$: \text{Con} \rightarrow \text{Sort}$	$\multimap_p \eta : ((p_p \circ \gamma)_{,p} (\mathbf{q}_p [\gamma]^P)) = \gamma$	
$\multimap [-]^t$	$: \text{Tm } \Gamma \rightarrow \text{Sub } \Delta \Gamma \rightarrow \text{Tm } \Delta$	$\multimap [-]^P : \text{Pf } \Gamma K \rightarrow (\gamma : \text{Sub } \Delta \Gamma) \rightarrow \text{Pf } \Delta (K[\gamma]^F)$	
$[\text{id}]^t$	$: t [\text{id}]^t = t$		
$[\circ]^t$	$: t [\gamma \circ \delta]^t = (t [\gamma]^t) [\delta]^t$		

We omit the formula connectives and their intro/elim rules.

5.3 Splitting the context

In our current setting contexts work as so called telescopes, that means that later things in the context can depend on previous things. For example when we want to prove the formula $\text{Eq } t t' \supset \text{Eq } t' t$ then we would be searching for an element of $\text{Pf} (\diamond \multimap_t \multimap_t) (\text{Eq } q (q[p]) \supset \text{Eq } (q[p]) q)$.

Example 5 (Proof of $\text{Pf} (\diamond \multimap_t \multimap_t) (\text{Eq } q (q[p]) \supset \text{Eq } (q[p]) q)$).

$$\begin{aligned}
 & \text{Pf} (\diamond \multimap_t \multimap_t) (\text{Eq } q (q[p]) \supset \text{Eq } (q[p]) q) && = \\
 & && \supset \text{intro} \\
 & \text{Pf} (\diamond \multimap_t \multimap_t \multimap_p (\text{Eq } q (q[p]))) (\text{Eq } (q[p]) q) && = \\
 & && \text{subst}' (\text{Eq } q (q[p])) q \\
 & \text{Pf} (\diamond \multimap_t \multimap_t \multimap_p (\text{Eq } q (q[p]))) && = \\
 & && \text{refl} \\
 & && \square
 \end{aligned}$$

We start with using the introduction rule for implication, after that our context becomes $(\diamond \triangleright_t \triangleright_t \triangleright_p (\mathbf{Eq} \mathbf{q} (\mathbf{q}[t]^p)))$. Here we can see that the third item in the context is a proof of $\mathbf{Eq} \mathbf{q} (\mathbf{q}[t]^p)$ so of $\mathbf{Eq} t t'$, this only makes sense because t and t' are also in the context and we refer to them using $\mathbf{q}$ and $\mathbf{p}$. This dependence is only in one direction, elements of $\mathbf{Pf}$ so proof variables can depend on term, but not the other way around. This could suggest there might be a way to separate the term part of the context and the proof part. In general when working with a GAT that is a result of the SOGAT to GAT translation its not possible to separate the context into a part for each of the sorts. In our scenario it is possible to split the context while still preserving some kind of equivalence between the 2 notions of models. We conjecture this correspondence to be the following

Conjecture 1 (Correspondence between split and non-split FOL GAT). *There is an adjunction between the category of models of split FOL and non-split FOL. The left adjoint splits the context into two parts and the right adjoint glues them back together. The right adjoint preserves the initial object (syntax).*

$$\begin{array}{ccc} \text{Mod}(\text{Fol}_{\text{split}}) & \xrightleftharpoons{\quad \perp \quad} & \text{Mod}(\text{Fol}) \end{array}$$

The reason why the two presentations are not the same is because in the split version we lose the ability for the dependency, while we certainly think of models of first-order logic as ones where terms don't depend on proofs, in the non-split case this possibility is still there, there can be models where they do depend on it. In the split case we removed this ability so we lose some models but the fact that the initial model is preserved means that in some sense we didn't lose much, we didn't lose the so called "intended" models. The main reason why we work with the split context version is because defining the iterator for the non-split case is very hard and involves lots of transporting over previous equalities, this is because of the dependence. We show how the split presentation looks like, here we make explicit the dependencies by making term contexts, and a proof contexts which depend on term contexts.

Definition 8 $(\text{Fol}_{\text{split}} (\mathcal{U}))$.

$$\begin{array}{ll} \text{Con}_t & : \text{Sort} \\ \text{Sub}_t & : \text{Con}_t \rightarrow \text{Con}_t \rightarrow \text{Sort} \end{array} \qquad \begin{array}{l} [\circ_t]^F : \Gamma [\gamma \circ_t \delta]^F = (A [\gamma]^F) [\delta]^F \\ \text{Tm} : \text{Con}_t \rightarrow \text{Sort} \end{array}$$

id_t	$: \text{Sub}_t \Gamma \Gamma$	$-[-]^t : \text{Tm } \Gamma \rightarrow \text{Sub}_t \Delta \Gamma \rightarrow \text{Tm } \Delta$
$- \circ_t -$	$: \text{Sub}_t \Delta \Gamma \rightarrow \text{Sub}_t \Theta \Delta \rightarrow \text{Sub}_t \Theta \Gamma$	$[\text{id}_t]^t : t [\text{id}_t]^t = t$
idl_t	$: \text{id}_t \circ_t \gamma = \gamma$	$[\circ_t]^t : t [\gamma \circ_t \delta]^t = (t [\gamma]^t) [\delta]^t$
idr_t	$: \gamma \circ_t \text{id}_t = \gamma$	$-\triangleright_t : \text{Con}_t \rightarrow \text{Con}_t$
assoc_t	$: (\gamma \circ_t \delta) \circ_t \theta = \gamma \circ_t (\delta \circ_t \theta)$	$-,_t - : \text{Sub}_t \Delta \Gamma \rightarrow \text{Tm } \Delta \rightarrow \text{Sub}_t \Delta (\Gamma \triangleright_t)$
$\diamond_t$	$: \text{Con}_t$	$\mathbf{p}_t : \text{Sub}_t (\Gamma \triangleright_t) \Gamma$
ϵ_t	$: \text{Sub}_t \Gamma \diamond_t$	$\mathbf{q}_t : \text{Tm } (\Gamma \triangleright_t)$
$\diamond_t \eta$	$: (\sigma : \text{Sub}_t \Gamma \diamond_t) \rightarrow \sigma = \epsilon_t$	$\triangleright_t \beta_1 : (\mathbf{p}_t \circ_t (\gamma ,_t t)) = \gamma$
Con_p	$: \text{Con}_t \rightarrow \text{Sort}$	$\triangleright_t \beta_2 : (\mathbf{q}_t [\gamma ,_t t]^t) = t$
subpisprop	$: (\gamma \delta : \text{Sub}_p \Delta \Gamma) \rightarrow \gamma = \delta$	$\triangleright_t \eta : ((\mathbf{p}_t \circ_t \gamma) ,_t (\mathbf{q}_t [\gamma]^t)) = \gamma$
$-[-]^C$	$: \text{Con}_p \Gamma \rightarrow \text{Sub}_t \Delta \Gamma \rightarrow \text{Con}_p \Delta$	$\text{Pf} : \text{Con}_p \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{Sort}$
$[\text{id}_t]^C$	$: \Gamma [\text{id}_t]^C = \Gamma$	$\text{pfisprop} : (k \ l : \text{Pf } \Delta \ K) \rightarrow k = l$
$[\circ_t]^C$	$: \Gamma [\gamma \circ_t \delta]^C = (\Gamma [\gamma]^C) [\delta]^C$	$-[-]^P : \text{Pf } \Gamma \ K \rightarrow \text{Sub}_t \Delta \Gamma \rightarrow \text{Pf } (\Gamma [\gamma]^C) (K [\gamma]^F)$
Sub_p	$: \text{Con}_p \Gamma \rightarrow \text{Con}_p \Gamma \rightarrow \text{Sort}$	$-[-]^P : \text{Pf } \Gamma \ K \rightarrow \text{Sub}_p \Delta \Gamma \rightarrow \text{Pf } \Delta \ K$
$- \circ_p -$	$: \text{Sub}_p \Delta \Gamma \rightarrow \text{Sub}_p \Theta \Delta \rightarrow \text{Sub}_p \Theta \Gamma$	$- \triangleright_p - : \text{Con}_p \Gamma \rightarrow \text{For } \Gamma \rightarrow \text{Con}_p \Gamma$
id_p	$: \text{Sub}_p \Gamma \Gamma$	$-,_p - : (\gamma : \text{Sub}_p \Delta \Gamma) \rightarrow \text{Pf } \Delta \ K \rightarrow$
$\diamond_p$	$: \text{Con}_p \Gamma$	$\text{Sub}_p \Delta (\Gamma \triangleright_p K)$
ϵ_p	$: \text{Sub}_p \Gamma \diamond_p$	$\mathbf{p}_p : \text{Sub}_p (\Gamma \triangleright_p K) \Gamma$
For	$: \text{Con}_t \rightarrow \text{Sort}$	$\mathbf{q}_p : \text{Pf } (\Gamma \triangleright_p K) \ K$
$-[-]^F$	$: \text{For } \Gamma \rightarrow \text{Sub}_t \Delta \Gamma \rightarrow \text{For } \Delta$	$\diamond_p [] : \diamond_p [\gamma]^C = \diamond_p$
$[\text{id}_t]^F$	$: A [\text{id}_t]^F = A$	$\triangleright_p [] : (\Gamma \triangleright_p K) [\gamma]^C = (\Gamma [\gamma]^C) \triangleright_p (K [\gamma]^F)$
$[\circ_t]^F$	$: A [\gamma \circ_t \delta]^F = (A [\gamma]^F) [\delta]^F$	

We omit the formula connectives and their intro/elim rules. Proof context can now also be substituted by a term substitution, this is done by $-[-]^C$. Proofs can be substituted in two ways, via $-[-]^P$ which only substitutes the term part or by $-[-]^F$ which only does the proof part. The equation **pfisprop** expresses that elements of proofs for a formula are equal, meaning there is at most one way to prove a formula. The equation **subpisprop** says that same for proof substitution s. This is why the equations that we have for **Sub_t** are missing for **Sub_p**. The equations $\diamond_p []$ and $\triangleright_p []$ express how to substitute **Con_p**-s.

Chapter 6

Syntax as a definable quotient

In this section we describe how we can define the syntax of $\mathbf{Fol}_{\text{split}}$. The syntax of a GAT is the initial object in the category of models. This characterisation means that the syntax is defined by its ability to be interpreted in any other model, that is what it means that there is a model homomorphism from the syntax (initial object) to any other model. This homomorphism is called the iterator or eliminator of the syntax. To define the syntax in a quotient free way, it needs to be definable as an inductive-inductive type [16]. Inductive-inductive types allow multiple types to be defined at the same time so they are more expressive than inductive types. Then the equations of the syntax hold because we define the substitution operations recursively on this induction-inductive type.

6.1 Syntax as a IIT

The syntax is given as an inductive-inductive type $(\mathcal{U})$. We explain the main elements and show some code fragments.

Term context We define term contexts as inductively by the empty context and context extension.

```
data ConTm : Set where
  ◇t : ConTm
  _▷t : ConTm → ConTm
```

Terms We first define term variables **Var** mutually with their substitutions. They are De Bruijn indices, the constructor vz is the zero De Bruijn index and vs is the successor.

```
data Var : ConTm → Set where
  vz : ∀{Γ} → Var (Γ ▷t)
  vs : ∀{Γ} → Var Γ → Var (Γ ▷t)

data VarSub : ConTm → ConTm → Set where
  et : ∀{Δ} → VarSub Δ ◊t
  _,t_ : ∀{Γ Δ} → VarSub Δ Γ → Var Δ → VarSub Δ (Γ ▷t)
```

Terms are mutually inductively defined with term vectors **Tms**. A term in a context Γ is either a term variable indexing into the term context **Var** or a function symbol with arity n and term vector that has length n . Term vectors are lists of terms that carry their length in their type.

```
data Tm (Γ : ConTm) : Set where
  var : V.Var Γ → Tm Γ
  fun : (n : ℕ) → funar n → Tms Γ n → Tm Γ

Tms : ConTm → ℕ → Set
Tms Γ zero = 1
Tms Γ (suc n) = Tms Γ n × Tm Γ
```

Term substitutions then are just lists of terms indexed by the two contexts.

```
data Subt : ConTm → ConTm → Set where
  et : ∀{Δ} → Subt Δ ◊t
  _,t_ : ∀{Γ Δ} → Subt Δ Γ → Tm Δ → Subt Δ (Γ ▷t)
```

Formulas Formulas are defined inductively indexed by the term context they are in. The quantifiers $\forall'$ and $\exists'$ take a formula in an extended term context introducing a new term variable. The constructor **rel** allows forming formulas from relation symbols. Substitution on formulas is defined in a recursive way.

```
data For (Γt : ConTm) : Set where
  ⊥ : For Γt
```

```

T      : For  $\Gamma t$ 
_▷_    : For  $\Gamma t \rightarrow$  For  $\Gamma t \rightarrow$  For  $\Gamma t$ 
_∧_    : For  $\Gamma t \rightarrow$  For  $\Gamma t \rightarrow$  For  $\Gamma t$ 
_∨_    : For  $\Gamma t \rightarrow$  For  $\Gamma t \rightarrow$  For  $\Gamma t$ 
∀'     : For  $(\Gamma t \triangleright t) \rightarrow$  For  $\Gamma t$ 
∃'     : For  $(\Gamma t \triangleright t) \rightarrow$  For  $\Gamma t$ 
Eq     : Tm  $\Gamma t \rightarrow$  Tm  $\Gamma t \rightarrow$  For  $\Gamma t$ 
rel    :  $(n : \mathbb{N}) \rightarrow relar\ n \rightarrow$  Tms  $\Gamma t\ n \rightarrow$  For  $\Gamma t$ 
    
```

Proof contexts Proof contexts are defined similarly to term context but they now include a formula in the context extension operation encoding what formula is proved by the proof variable. The $-[-]^C$ context substitution operation is defined recursively.

```

data ConPf ( $\Gamma t : \text{ConTm}$ ) : Set where
  ◇p : ConPf  $\Gamma t$ 
  _▷p_ : ConPf  $\Gamma t \rightarrow$  For  $\Gamma t \rightarrow$  ConPf  $\Gamma t$ 

_[]^C :  $\forall \{\Gamma t\} \rightarrow$  ConPf  $\Gamma t \rightarrow \forall \{\Delta t\} \rightarrow$  Subt  $\Delta t\ \Gamma t \rightarrow$  ConPf  $\Delta t$ 
◇p [  $\gamma$  ]^C = ◇p
( $\Gamma p \triangleright p\ K$ ) [  $\gamma$  ]^C = ( $\Gamma p$  [  $\gamma$  ]^C) ▷p ( $K$  [  $\gamma$  ]^F)
    
```

Proofs Proofs are defined mutually inductively with proof substitutions.

```

data Subp :  $\{\Gamma t : \text{ConTm}\} \rightarrow$  ConPf  $\Gamma t \rightarrow$  ConPf  $\Gamma t \rightarrow$  Prop
data Pf   :  $\{\Gamma t : \text{ConTm}\} \rightarrow$  ConPf  $\Gamma t \rightarrow$  For  $\Gamma t \rightarrow$  Prop

data Subp where
  εp :  $\forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow$  Subp  $\Gamma p$  ◇p
  idp :  $\forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow$  Subp  $\Gamma p\ \Gamma p$ 
  _◦p_ :  $\forall \{\Gamma t\} \{\Gamma p\ \Gamma p' \ \Gamma p'' : \text{ConPf } \Gamma t\} \rightarrow$ 
    Subp  $\Gamma p' \ \Gamma p \rightarrow$  Subp  $\Gamma p'' \ \Gamma p' \rightarrow$  Subp  $\Gamma p'' \ \Gamma p$ 
  pp :  $\forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \{K\} \rightarrow$ 
    Subp  $(\Gamma p \triangleright p\ K) \ \Gamma p$ 
  _.,p_ :  $\forall \{\Gamma t\} \{\Gamma p\ \Gamma p' : \text{ConPf } \Gamma t\} \{K\} \rightarrow$ 
    Subp  $\Gamma p' \ \Gamma p \rightarrow$  Pf  $\Gamma p' \ K \rightarrow$  Subp  $\Gamma p' \ (\Gamma p \triangleright p\ K)$ 
    
```

data Pf where

$$\begin{aligned}
 \text{exfalse} &: \forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \{K\} \rightarrow \text{Pf } \Gamma p \perp \rightarrow \text{Pf } \Gamma p K \\
 \text{tt} &: \forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \text{Pf } \Gamma p \top \\
 \supset \text{intro} &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } (\Gamma p \triangleright_p K) L \rightarrow \text{Pf } \Gamma p (K \supset L) \\
 _ \$ _ &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p (K \supset L) \rightarrow \text{Pf } \Gamma p K \rightarrow \text{Pf } \Gamma p L \\
 \wedge \text{intro} &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p K \rightarrow \text{Pf } \Gamma p L \rightarrow \text{Pf } \Gamma p (K \wedge L) \\
 \wedge \text{elim}_1 &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p (K \wedge L) \rightarrow \text{Pf } \Gamma p K \\
 \wedge \text{elim}_2 &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p (K \wedge L) \rightarrow \text{Pf } \Gamma p L \\
 \vee \text{intro}_1 &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p K \rightarrow \text{Pf } \Gamma p (K \vee L) \\
 \vee \text{intro}_2 &: \forall \{\Gamma t\} \{K L\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p L \rightarrow \text{Pf } \Gamma p (K \vee L) \\
 \vee \text{elim} &: \forall \{\Gamma t\} \{K L C\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } (\Gamma p \triangleright_p K) C \rightarrow \\
 &\quad \text{Pf } (\Gamma p \triangleright_p L) C \rightarrow \\
 &\quad \text{Pf } \Gamma p (K \vee L) \rightarrow \\
 &\quad \text{Pf } \Gamma p C \\
 \forall \text{intro} &: \forall \{\Gamma t\} \{K \Gamma p\} \rightarrow \\
 &\quad \text{Pf } \{\Gamma t \triangleright_t\} (\Gamma p \text{ [pt] C}) K \rightarrow \text{Pf } \{\Gamma t\} \Gamma p (\forall' K) \\
 \forall \text{elim} &: \forall \{\Gamma K \Gamma p\} \rightarrow \\
 &\quad \text{Pf } \{\Gamma\} \Gamma p (\forall' K) \rightarrow \text{Pf } \{\Gamma \triangleright_t\} (\Gamma p \text{ [pt] C}) K \\
 \exists \text{intro} &: \forall \{\Gamma t K\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 &\quad (t : \text{Tm } \Gamma t) \rightarrow \\
 &\quad \text{Pf } \Gamma p (K \text{ [idt ,t t] F}) \rightarrow \\
 &\quad \text{Pf } \Gamma p (\exists' K) \\
 \exists \text{elim} &: \forall \{\Gamma t\} \{K : \text{For } (\Gamma t \triangleright_t)\} \{\Gamma p : \text{ConPf } \Gamma t\} \{L : \text{For } \Gamma t\} \rightarrow \\
 &\quad \text{Pf } \Gamma p (\exists' K) \rightarrow \\
 &\quad \text{Pf } ((\Gamma p \text{ [pt] C}) \triangleright_p K \text{ [pt ,t qt] F}) (L \text{ [pt] F}) \rightarrow
 \end{aligned}$$

$$\begin{aligned}
 & \text{Pf } \Gamma p \ L \\
 \text{ref} & : \forall \{\Gamma t\} \{a\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \text{Pf } \Gamma p \ (\text{Eq } a \ a) \\
 \text{subst}' & : \forall \{\Gamma t\} (K : \text{For } (\Gamma t \triangleright t)) \{t \ t' : \text{Tm } \Gamma t\} \{\Gamma p\} \rightarrow \\
 & \quad \text{Pf } \Gamma p \ (\text{Eq } t \ t') \rightarrow \\
 & \quad \text{Pf } \Gamma p \ (K \ [\text{idt } , t] F) \rightarrow \\
 & \quad \text{Pf } \Gamma p \ (K \ [\text{idt } , t'] F) \\
 []p & : \forall \{\Gamma t\} \{K\} \{\Gamma p : \text{ConPf } \Gamma t\} \rightarrow \\
 & \quad \text{Pf } \Gamma p \ K \rightarrow \forall \{\Delta t : \text{ConTm}\} \rightarrow \\
 & \quad (\gamma : \text{Subt } \Delta t \ \Gamma t) \rightarrow \\
 & \quad \text{Pf } (\Gamma p \ [\gamma] C) \ (K \ [\gamma] F) \\
 []P & : \forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \{K : \text{For } \Gamma t\} \rightarrow \\
 & \quad \text{Pf } \Gamma p \ K \rightarrow \forall \{\Gamma p'\} \rightarrow \\
 & \quad \text{Subp } \Gamma p' \ \Gamma p \rightarrow \\
 & \quad \text{Pf } \Gamma p' \ K \\
 \text{qp} & : \forall \{\Gamma t\} \{\Gamma p : \text{ConPf } \Gamma t\} \{K : \text{For } \Gamma t\} \rightarrow \\
 & \quad \text{Pf } (\Gamma p \triangleright_p K) \ K
 \end{aligned}$$

6.2 Iterator

We describe the notion of model homomorphism for $\text{Fol}_{\text{Split}}$ and show the iterator which is a morphism from the syntax to any model.

Giving the iterator with the split notion of model is straight forward. Sorts are defined recursively on the syntax so we iterate over them and send them to the appropriate operations.

$$\begin{aligned}
 \ulcorner \diamond t \urcorner \neg \text{Cont} &= \diamond t \\
 \ulcorner \Gamma t \triangleright t \urcorner \neg \text{Cont} &= \ulcorner \Gamma t \urcorner \neg \text{Cont} \triangleright t \\
 \ulcorner \epsilon t \urcorner \neg \text{Subt} &= M.\epsilon t \\
 \ulcorner \gamma \urcorner , t \urcorner \neg \text{Subt} &= \ulcorner \gamma \urcorner \neg \text{Subt } M., t \ulcorner t \urcorner \neg \text{Tm} \\
 \ulcorner V.vz \urcorner \neg \text{Var} &= M.qt \\
 \ulcorner V.vs \ t \urcorner \neg \text{Var} &= \ulcorner t \urcorner \neg \text{Var } M.[M.pt]t \\
 \ulcorner \text{var } t \urcorner \neg \text{Tm} &= \ulcorner t \urcorner \neg \text{Var}
 \end{aligned}$$

$$\begin{aligned}
 \ulcorner \text{fun } n \text{ a } ts \urcorner \neg \text{Tm} &= \text{M.fun } n \text{ a } \ulcorner ts \urcorner \neg \text{Tms} \\
 \ulcorner _ \urcorner \neg \text{Tms } \{\Gamma t\} \{\text{zero}\} * &= \text{M.}\epsilon s \\
 \ulcorner _ \urcorner \neg \text{Tms } \{\Gamma t\} \{\text{suc } n\} (ts, \Sigma t) &= \ulcorner ts \urcorner \neg \text{Tms } \text{M.},s \ulcorner t \urcorner \neg \text{Tm}
 \end{aligned}$$

Here M is an arbitrary model of Fol and I is the syntax. For formulas we have :

$$\begin{aligned}
 \ulcorner \perp \urcorner \neg \text{For} &= \text{M.}\perp \\
 \ulcorner \top \urcorner \neg \text{For} &= \text{M.}\top \\
 \ulcorner K \supset L \urcorner \neg \text{For} &= \ulcorner K \urcorner \neg \text{For } \text{M.}\supset \ulcorner L \urcorner \neg \text{For} \\
 \ulcorner K \wedge L \urcorner \neg \text{For} &= \ulcorner K \urcorner \neg \text{For } \text{M.}\wedge \ulcorner L \urcorner \neg \text{For} \\
 \ulcorner K \vee L \urcorner \neg \text{For} &= \ulcorner K \urcorner \neg \text{For } \text{M.}\vee \ulcorner L \urcorner \neg \text{For} \\
 \ulcorner \forall' K \urcorner \neg \text{For} &= \text{M.}\forall' \ulcorner K \urcorner \neg \text{For} \\
 \ulcorner \exists' K \urcorner \neg \text{For} &= \text{M.}\exists' \ulcorner K \urcorner \neg \text{For} \\
 \ulcorner \text{Eq } t \text{ } t' \urcorner \neg \text{For} &= \text{M.Eq } \ulcorner t \urcorner \neg \text{Tm } \ulcorner t' \urcorner \neg \text{Tm} \\
 \ulcorner \text{rel } n \text{ a } ts \urcorner \neg \text{For} &= \text{M.rel } n \text{ a } \ulcorner ts \urcorner \neg \text{Tms}
 \end{aligned}$$

Because the operations are defined recursively in the syntax we don't have much problem with giving the iterator. We show some of the equations that the operations are preserved.

The following code says that term substitution composition is preserved by the iterator. We prove this statement by induction on γ_t . Because term composition in the syntax was defined by pattern matching on γ_t , it makes our goal compute and the two cases reduce to some easy equations. The first case needs that:

$$\epsilon_{t_M} = \epsilon_{t_M} \circ_{t_M} \ulcorner \delta_t \urcorner \neg \text{Sub}_t$$

and we know that any substitution that goes from $\diamond_{t_M}$ to some Γ has to be ϵ_{t_M} , so this case is proved by $\diamond_t \eta_M : \sigma = \epsilon_{t_M}$ and symmetry.

$$\begin{aligned}
 \ulcorner \text{ot} \urcorner : \forall \{\Gamma t \Delta t \Theta t\} \{\gamma t : \text{Subt } \Delta t \Gamma t\} \{\delta t : \text{Subt } \Theta t \Delta t\} \rightarrow \\
 \ulcorner \gamma t \text{ l.ot } \delta t \urcorner \neg \text{Subt} &\equiv \ulcorner \gamma t \urcorner \neg \text{Subt } \text{M.ot } \ulcorner \delta t \urcorner \neg \text{Subt} \\
 \ulcorner \text{ot} \urcorner \{\diamond t\} \{\Delta t\} \{\Theta t\} \{\epsilon t\} \{\delta t\} &= \text{sym } (\text{M.}\diamond t \eta (\text{M.}\epsilon t \text{ M.ot } \ulcorner \delta t \urcorner \neg \text{Subt})) \\
 \ulcorner \text{ot} \urcorner \{\Gamma t \triangleright t\} \{\Delta t\} \{\Theta t\} \{\gamma t, t\} \{\delta t\} &= \\
 \text{trans } (\text{cong } (\lambda z \rightarrow \text{proj}_1 z \text{ M.},t \text{ proj}_2 z) \\
 (\text{mk}, = (\ulcorner \text{ot} \urcorner \{\gamma t = \gamma t\} \{\delta t = \delta t\}) (\ulcorner t \urcorner \neg \{t = t\}))) \\
 (\text{sym } \text{M.},\text{tot})
 \end{aligned}$$

The second case is a little more involved so we explain it step by step. We will need the lemma $(_,t \circ_t)_M : (\gamma_t, t) \circ_t \delta_t = (\gamma_t \circ_t \delta_t), t (t[\delta_t]^t)$ which says how $-,t-$

interacts with $- \circ_t -$. We first rewrite the left hand side under $-_{,t} -$ by a recursive call to $\ulcorner \circ_t \urcorner$ and with $\ulcorner []^t \urcorner : \ulcorner t[\gamma_t]^t \urcorner \text{Tm} \equiv \ulcorner t \urcorner \text{Tm } M. [\ulcorner \gamma_t \urcorner \text{Sub}_t]^t$

$\ulcorner \circ_t \urcorner$ *second case*.

$$\begin{aligned}
 \ulcorner \gamma_t \circ_t \delta_t \urcorner \text{Sub}_t \text{ } M. ,_t \ulcorner t[\delta_t]^t \urcorner \text{Tm} &= \\
 \text{cong-bin } (M. ,_t) (\ulcorner \circ_t \urcorner) (\ulcorner []^t \urcorner) &= \\
 (\ulcorner \gamma_t \urcorner \text{Sub}_t \text{ } M. \circ_t \ulcorner \delta_t \urcorner \text{Sub}_t) \text{ } M. ,_t \ulcorner t \urcorner \text{Tm } M. [\ulcorner \delta_t \urcorner \text{Sub}_t]^t &= \\
 \text{sym } (M. ,_t \circ_t) &= \\
 \ulcorner \gamma_t \urcorner \text{Sub}_t \text{ } M. \circ_t \ulcorner \delta_t \urcorner \text{Sub}_t &= \\
 \text{refl} &=
 \end{aligned}$$

□

$$\begin{aligned}
 \ulcorner \epsilon_p \urcorner \text{Sub}_p &= M. \epsilon_p \\
 \ulcorner \gamma ,_p \text{PfK} \urcorner \text{Sub}_p &= \ulcorner \gamma \urcorner \text{Sub}_p \text{ } M. ,_p \ulcorner \text{PfK} \urcorner \text{Pf} \\
 \ulcorner \text{idp} \urcorner \text{Sub}_p &= M. \text{idp} \\
 \ulcorner \gamma \circ_p \delta \urcorner \text{Sub}_p &= \ulcorner \gamma \urcorner \text{Sub}_p \text{ } M. \circ_p \ulcorner \delta \urcorner \text{Sub}_p \\
 \ulcorner \text{pp} \urcorner \text{Sub}_p &= M. \text{pp}
 \end{aligned}$$

Sub_p and Pf is straight forward, except the **intro** and **elim** rules for the $\forall$, $\exists$ and **Eq**. In these cases we have to use some transport, we show one example. The case for the $\forall$ intro is the following:

$$\begin{aligned}
 \ulcorner \forall \text{intro} \urcorner \{ \Gamma t \} \{ K \} \{ \Gamma \} P_K \urcorner \text{Pf} &= \\
 \text{let } P_K' &= \\
 \text{substp} &= \\
 (\lambda z \rightarrow M. \text{Pf } z \ulcorner K \urcorner \text{For}) &= \\
 (\text{trans} &= \\
 \ulcorner []^C \urcorner &= \\
 (\text{cong } (\lambda z \rightarrow \ulcorner \Gamma \urcorner \text{Conp } M. [z] \text{C}) \ulcorner \text{pt} \urcorner)) &= \\
 \ulcorner P_K \urcorner \text{Pf} &= \\
 \text{in } M. \forall \text{intro } P_K' &=
 \end{aligned}$$

What we would want to write is $\forall \text{intro}_M \ulcorner P_K \urcorner \text{Pf}$ but that doesn't typecheck because $\ulcorner P_K \urcorner \text{Pf}$ has type $\text{Pf}_M (\ulcorner \Gamma [\text{p}_t]^C \urcorner \text{Con}_p) (\ulcorner K \urcorner \text{For})$ while $\forall \text{intro}_M$ want a term

of type $\text{Pf}_M (\ulcorner \Gamma \urcorner \text{Con}_p [\mathbf{p}_t]^C) (\ulcorner K \urcorner \text{For})$. Because of this $\ulcorner P_K \urcorner \text{Pf}$ has to be transported along this equality with **substp**. So we want to prove that

$$\ulcorner \Gamma [\mathbf{p}_t]^C \urcorner \text{Con}_p = \ulcorner \Gamma \urcorner \text{Con}_p [\mathbf{p}_{t_M}]_M^C$$

$\forall$ intro **substp** *proof*.

$$\begin{aligned} \ulcorner \Gamma [\mathbf{p}_t]^C \urcorner \text{Con}_p &= \\ &\ulcorner []^C \urcorner \\ \ulcorner \Gamma \urcorner \text{Con}_p [\ulcorner \mathbf{p}_t \urcorner \text{Sub}_t]_M^C &= \\ &\ulcorner \mathbf{p}_t \urcorner \\ \ulcorner \Gamma \urcorner \text{Con}_p [\mathbf{p}_{t_M}]_M^C &= \\ &\text{refl} \end{aligned}$$

□

The equalities we use in this proof $\ulcorner []^C \urcorner$ and $\ulcorner \mathbf{p}_t \urcorner$ say how $\mathbf{p}_t$ and $-[-]^C$ interact with the iterator. These equations are proven while defining the iterator. There are similar proofs for all the operations.

$$\begin{aligned} \ulcorner []^C \urcorner \{ \Gamma t \} \{ \Delta t \} \{ \diamond p \} \{ \gamma t \} &= \text{sym } M. \diamond p [] \\ \ulcorner []^C \urcorner \{ \Gamma t \} \{ \Delta t \} \{ (\Gamma \triangleright p K) \} \{ \gamma t \} &= \\ \text{trans} & \\ (\text{cong } (\lambda z \rightarrow \text{proj}_1 z \text{ } M. \triangleright p \text{ } \text{proj}_2 z)) & \\ (\text{mk}, = & \\ (\ulcorner []^C \urcorner \{ \Gamma = \Gamma \}) & \\ (\ulcorner []^F \urcorner \{ K = K \})) & \\ (\text{sym } M. \triangleright p []) & \end{aligned}$$

Chapter 7

Completeness for Propositional logic

In this chapter we describe what we mean by a completeness result, and show completeness of Kripke (Presheaf) models.

7.1 Completeness

Completeness and soundness are important properties of models of logical systems, they relate syntax and semantics, meaning they express the relation of a particular class of models to the syntax of the theory. Completeness and soundness can then be expressed as:

$$\begin{aligned}\text{Completeness} &: \text{Pf} \vdash \Gamma \neg^C \vdash A \neg^F \rightarrow \text{Pf}_I \vdash A \\ \text{Soundness} &: \text{Pf}_I \vdash A \rightarrow \text{Pf} \vdash \Gamma \neg^C \vdash A \neg^F\end{aligned}$$

Here Pf_I is a sort of the syntax and the normal Pf is a sort in an arbitrary model M . We then say that model M is complete/sound for some theory T . For a set of models to be complete/sound for the theory T , we need that

- Completeness : If a formula is true in all models in the set, then it is provable
- Soundness : If a formula is provable, then it true in all models in the set

Soundness expresses that anything which can be proven by syntactic means (provable in the syntax) has a proof in the semantics (in the chosen model/models).

Completeness is the inverse, that is if something had a semantic proof then it has to have a syntactic one.

7.2 Kripke semantics

The traditional description of Kripke models can be found in Dirk van Dalen's book on logic [1]. Kripke semantics is often presented in the setting of modal logic, but it also gives semantics to intuitionistic logic. The main idea of Kripke semantics is the notion of "growth of knowledge". Formulas are interpreted in a way that they are indexed by a preorder of potential worlds. This interpretation has to satisfy a monotonicity property that expresses that as we go to later worlds we don't lose any knowledge but we can gain more. This means that we can interpret formulas by a set valued function, over some preorder W , and an upwards closed set D such that

$$l \geq k \rightarrow (k \in D \Rightarrow l \in D)$$

Definition 9 (Presheaf). A presheaf P on a category $\mathcal{C}$ is a contravariant functor from $\mathcal{C}$ to the category of Sets. It has the following components:

- Action on objects, denoted by

$$|P| : \mathcal{C} \rightarrow \text{Set}$$

- Action on morphisms, denoted by

$$x \langle f \rangle_P : |P| I \rightarrow \text{Hom}(J, I) \rightarrow |P| J$$

- Proofs that it preserves identity and composition

$$\begin{aligned} \langle \text{id} \rangle_P &: x \langle \text{id}_I \rangle = x \\ \langle \circ \rangle_P &: x \langle f \circ g \rangle_P = (x \langle f \rangle_P) \langle g \rangle_P \end{aligned}$$

Definition 10 (Proof irrelevant presheaf/Upper set). A proof irrelevant presheaf P on a category $\mathcal{C}$, also called an upwards closed set is a presheaf such that

$$\forall I. \forall (x y : |P| I). x = y$$

This means we don't need to state $\langle \text{id} \rangle_P$ and $\langle \circ \rangle_P$.

Kripke semantics is a proof irrelevant version of a presheaf semantics. We give the interpretation of every component of $\mathbf{Prop}_{GAT}^I$ in the Kripke semantics.

Definition 11 (Kripke semantics for $\mathbf{Prop}_{GAT}^I$ ($\mathcal{U}$)). Let $\mathcal{C}$ be an arbitrary preorder (thin category) and A_{pv} an upper set for any atomic formula A .

Contexts and formulas are interpreted by a proof irrelevant presheaf over $\mathcal{C}$, that is an upwards closed set P . The empty context $\diamond$ and $\top$ are interpreted as the trivial upper set, where $\mathbf{1}$ is the one elements set $\{*\}$.

$$\begin{aligned} |\mathbf{1U}|_w &:= \mathbf{1} \\ * \langle f \rangle_{\mathbf{1U}} &:= * \end{aligned}$$

Context extension and conjunction are interpreted by the product of two upper sets $- \times_{\mathbf{U}} -$

$$\begin{aligned} |A \times_{\mathbf{U}} B|_w &:= |A|_w \times |B|_w \\ (A_w, B_w) \langle f \rangle_{(A \times_{\mathbf{U}} B)} &:= (A_w \langle f \rangle_A, B_w \langle f \rangle_B) \end{aligned}$$

$\perp$ is the empty upper set, disjunction is the coproduct of two upper sets. Implication is interpreted by the following construction on upper sets.

$$\begin{aligned} |A \supset_{\mathbf{U}} B|_w &:= \forall w' \rightarrow w' \geq w \rightarrow |A|_{w'} \rightarrow |B|_{w'} \\ (K \langle f \rangle)_{A \supset_{\mathbf{U}} B} g w' A_{w'} &:= K (f \circ_{\mathcal{C}} g) A_{w'} \end{aligned}$$

Substitutions and proofs are interpreted by the following definition.

$$\begin{aligned} \text{Sub } \Gamma \Delta &:= \forall w \rightarrow |\Gamma|_w \rightarrow |\Delta|_w \\ \text{Pf } \Gamma A &:= \forall w \rightarrow |\Gamma|_w \rightarrow |A|_w \end{aligned}$$

Traditional this is written as

$$w \Vdash \Gamma \rightarrow w \Vdash A$$

Atoms are interpreted by their upper set A_{pv} .

7.3 Soundness and Completeness for Kripke models

Soundness is proven very easily in our presentation. In the category of models of $\mathbf{Prop}_{GAT}^I$ the initiality of the syntax guarantees that for any other model we have a map

$$\mathbf{Pf}_I \Gamma A \rightarrow \mathbf{Pf} \ulcorner \Gamma \urcorner^C \ulcorner A \urcorner^F$$

Soundness of Kripke models of $\mathbf{Prop}_{GAT}^I$ ($\hookrightarrow$). Soundness is given by the $\mathbf{Pf}$ component of the iterator.

$$\ulcorner - \urcorner^{\mathbf{Pf}} : \mathbf{Pf}_I \Gamma A \rightarrow \mathbf{Pf} \ulcorner \Gamma \urcorner^C \ulcorner A \urcorner^F$$

□

This proof actually works for any model, meaning that what we call model in our presentation is always sound. We now turn to completeness. Classically Kripke models are complete for intuitionistic logic with all connectives. This proof uses Zorn's lemma which is classically accepted but is not provable in constructive foundations. Our metatheory is a constructive one, type theory, so the classical proof doesn't work. If we drop some connectives and only keep the negative ones, namely $\top$, $- \wedge -$, $- \supset -$, then we can prove completeness. We call this positive fragment $\mathbf{Prop}_{Pos}^I$.

We prove this with a method similar to one used in the metatheory of type theory for proving normalisation, called normalisation by evaluation [17, 18]. Completeness asks of us to prove that if something is true in any Kripke model then its true in the syntax, we do this by picking a Kripke model over a specific preorder and upper set for the atomic formulas.

We chose the preorder of contexts $\mathbf{Con}_I$ and substitutions $\mathbf{Sub}_I \Delta \Gamma$ of the syntax. For any atom a and context an element of the preorder Γ (a context) we chose the upper set over a to be

$$\mathbf{Pf}_I \Gamma (\mathbf{atom}_I a)$$

which is upwards closed because of proof substitution. Then we define three functions

$$\begin{aligned} \text{reify} & : \forall(\Gamma : \text{Con})(A : \text{For}_I) \rightarrow |\ulcorner A^{\neg F} \urcorner| \Gamma \rightarrow \text{Pf}_I \Gamma A \\ \text{reflect-Con} & : \forall(\Gamma \Delta : \text{Con}) \rightarrow \text{Sub}_I \Gamma \Delta \rightarrow |\ulcorner \Delta^{\neg C} \urcorner| \Gamma \\ \text{reflect} & : \forall(\Gamma : \text{Con})(A : \text{For}_I) \rightarrow \text{Pf}_I \Gamma A \rightarrow |\ulcorner A^{\neg F} \urcorner| \Gamma \end{aligned}$$

mutually recursively. The functions **reify** and **reflect** carry out the translation between syntactic and semantic proofs. **reify** reifies a semantic proof into a syntactic one, and **reflect** reflects a syntactic proof into a semantic one. The function **reflect-Con** is the most straight forward one, it is defined via induction on Δ .

$$\begin{aligned} \text{reflect-Con} & : \forall(\Gamma \Delta : \text{Con}) \rightarrow \text{Sub}_I \Gamma \Delta \rightarrow |\ulcorner \Delta^{\neg C} \urcorner| \Gamma \\ \text{reflect-Con } \Gamma \diamond_I \gamma & := * \\ \text{reflect-Con } \Gamma (\Delta \triangleright_I x) \gamma & := (\text{reflect-Con } \Gamma \Delta (\text{p}_I \circ_I \gamma)), (\text{reflect } \Gamma x (\text{q}_I [\gamma]_I)) \end{aligned}$$

It reflects everything in the context via **reflect**. The function **reflect** is also defined via induction on $A : \text{For}$.

$$\begin{aligned} \text{reflect} & : \forall(\Gamma : \text{Con})(A : \text{For}_I) \rightarrow \text{Pf}_I \Gamma A \rightarrow |\ulcorner A^{\neg F} \urcorner| \Gamma \\ \text{reflect } \Gamma \top_I l & := * \\ \text{reflect } \Gamma (A \supset_I B) l & := \gamma a \mapsto \text{reflect } \Gamma B ((\supset \text{elim}_I l) [\gamma, \text{reify } \Gamma A a]_I) \\ \text{reflect } \Gamma (A \wedge_I B) l & := (\text{reflect } \Gamma A (\wedge \text{elim}_{1_I} l)), (\text{reflect } \Gamma B (\wedge \text{elim}_{2_I} l)) \\ \text{reflect } \Gamma (\text{atom}_I a) l & := l \end{aligned}$$

The strategy is to use the eliminator of the connective and recursive calls. We show the case for $\supset$, here we have to use both **reflect** and **reify** this is why they have to be defined mutually recursively.

reflect – $\supset$ –.

$$\begin{aligned} |\ulcorner B^{\neg F} \urcorner| w' & = \\ & \text{reflect } B \\ \text{Pf}_I w' B & = \\ & \text{—}[\gamma, \text{reify } A a]_I \\ \text{Pf}_I (\Gamma \triangleright_I A) B & = \end{aligned}$$

$$\text{Pf}_I \Gamma (A \supset_I B) \quad \begin{array}{c} \supset_I \text{ elim} \\ = \\ l \end{array}$$

□

In reify we use the introduction rules.

$$\begin{aligned} \text{reify} & : \forall (\Gamma : \text{Con})(A : \text{For}_I) \rightarrow |\ulcorner A \urcorner^F| \Gamma \rightarrow \text{Pf}_I \Gamma A \\ \text{reify } \Gamma \top_I l & := \top \text{ intro}_I \\ \text{reify } \Gamma (A \supset_I B) l & := \supset \text{ intro}_I (\text{reify } \Gamma B (l \text{ p}_I (\text{reflect } \Gamma A \text{ q}_I))) \\ \text{reify } \Gamma (A \wedge_I B) (l_a, l_b) & := \wedge \text{ intro}_I (\text{reify } \Gamma A l_a)(\text{reify } \Gamma B l_b) \\ \text{reify } \Gamma (\text{atom}_I a) l & := l \end{aligned}$$

We show the case for $\supset$ here also.

reify – $\supset$ –.

$$\begin{aligned} \text{Pf}_I \Gamma (A \supset B) & = \\ & \supset \text{ intro}_I \\ \text{Pf}_I (\Gamma \triangleright_I A) B & = \\ & \text{reify } B \\ |\ulcorner B \urcorner^F| (\Gamma \triangleright_I A) & = \\ & l \text{ p}_I \\ |\ulcorner A \urcorner^F| (\Gamma \triangleright_I A) & = \\ & \text{reflect } A \text{ q}_I \end{aligned}$$

□

Completeness of Prop_{Pos}^I ($\mathcal{U}$).

$$\text{completeness} : \forall A \rightarrow \text{Pf } \ulcorner \Gamma \urcorner^C \ulcorner A \urcorner^F \rightarrow \text{Pf}_I \Gamma A$$

is given by the term

$$\{\Gamma\} A f \mapsto \text{reify } A (f (\text{reflect-Con } \Gamma \text{ id}_I))$$

□

Completeness is given by the composition of **reflect-Con** and **reify**. We first reflect the syntactic context Γ into a semantic one without substituting, hence the id_I . We then use $f : \text{Pf } \ulcorner \Gamma \urcorner^C \ulcorner A \urcorner^F$ the semantic proof of A which is a function of type $\forall \Delta. \ulcorner \Gamma \urcorner^C \Delta \rightarrow \ulcorner A \urcorner^F \Delta$ so if given **reflect-Con** Γ $\text{id}_I : \ulcorner \Gamma \urcorner^C \Gamma$ it result in a term of type $\ulcorner A \urcorner^F \Gamma$. Then this can reified back into a syntactic proof $\text{Pf}_I \Gamma A$, which was our goal.

Chapter 8

Completeness Theorems for First-order logic

In this chapter we describe Beth or Sheaf semantics, and prove completeness of Beth models for Fol_{GAT}^I .

8.1 Beth semantics

Traditionally Beth semantics is defined in the context of trees and bar induction. We will take a generalisation of this view, and call Beth semantics the proof irrelevant version of sheaf semantics. Sheaf are topological generalisations of presheafs and the bar structure on Beth's trees form a sheaf [19].

To define Beth models we need a lot of new definitions. We first introduce the notion of a *sieve* and covers on sieves.

Definition 12 (Sieve). Let $\mathcal{C}$ be a category and I an object of $\mathcal{C}$. A sieve on I is a subfunctor of the representable functor $\text{Hom}(-, I)$. We use the notation $\text{Sieve } I$ for a sieve on I .

Unfolding the definition of a sieve we get the following component for a sieve R on I .

- A relation $\square_R$ between objects $J : \text{Ob}(\mathcal{C})$ and morphisms $f : \text{Hom}(J, I)$.
- such that this relation satisfies the following condition called restriction (**restr**):

$$\forall J K : \text{Ob}. \forall f : \text{Hom}(J, I). J \square_R f \rightarrow (g : \text{Hom}(K, J)) \rightarrow K \square_R (f \circ g)$$

We can also substitute on sieves, that is for a sieve R on I and $f : Hom(I, J)$ we have an operation $R[f]^s$ defined as

$$K \sqsubset_{R[f]^s} g := K \sqsubset_R (f \circ g)$$

Its easy to check that this definition satisfies the required property. Sieves allow us to defined the notion of a coverage, which we call *Grothendieck coverage* (Grothendieck topology).

Definition 13 (Grothendieck coverage). Let $\mathcal{C}$ be a category. A Grothendieck coverage on $\mathcal{C}$ is a relation between $I : Ob(\mathcal{C})$ and sieves R on I

$$I \triangleleft R$$

denoting that R is covering I . This covering relation has to satisfy:

- Substitution ($-[-]^c$) : if $I \triangleleft R$ then $\forall J : Ob(\mathcal{C})$ and $\forall f : Hom(J, I)$ we have $J \triangleleft (R[f]^s)$.
- Maximality (**maximal**) : if $I \sqsubset_R id_I$ then $I \triangleleft R$.
- Locality (**local**) : if $I \triangleleft R$ and if from

$$\forall J : Ob(\mathcal{C}). \forall f : Hom(J, I). J \sqsubset_R f$$

we can conclude

$$J \triangleleft S[f]^s$$

then $I \triangleleft S$.

We can then state what a sheaf is.

Definition 14 (Sheaf). A sheaf is a presheaf A on a category $\mathcal{C}$ with the following conditions

- *glue* or *sheaf condition* : $\forall I : Ob(\mathcal{C}). \forall R : \text{Sieve } I. I \triangleleft R \rightarrow (F : \forall J : Ob(\mathcal{C}). \forall f : Hom(J, I). J \sqsubset_R f \rightarrow |A| J) \rightarrow (\forall J K : Ob(\mathcal{C}). \forall g : Hom(J, I). \forall h : Hom(K, J). \forall r : Jq, \sqsubset_R F \mapsto (F J g r) \langle h \rangle_A = F K (g \circ h) (r[h]^s)) \rightarrow |A| I$
- an equations saying the naturality glue

- an equations expressing what happens when the fourth argument of glue is constant

where $|A| - : Ob \rightarrow Set$ is the action of A on objects. We denote A 's action on morphisms by $-\langle - \rangle_A : |A| I \rightarrow Hom(J, I) \rightarrow |A| J$ and the identity and composition preservation equations as $\langle id \rangle_A$ and $\langle \circ \rangle_A$.

The **gluing** property allows us to encode the topological natural of the positive connectives into our model, making the completeness proof possible. There is also a nice characterisation of Grothendieck coverages as Prop-valued monads in the internal language of the presheaf model of type theory, this is expored in Pierre-Marie Pédrot's online note Debunking Sheafs [20] and was used to get intuition for sheafs. We give a few variations which we will use to defined Beth models.

Definition 15 (Proof irrelevant sheaf). A proof irrelevant sheaf is a proof irrelevant presheaf A on a category $\mathcal{C}$ with the following conditions

- *glue or sheaf condition* : $\forall I : Ob(\mathcal{C}). \forall R : \text{Sieve } I. I \triangleleft R \rightarrow (\forall J : Ob(\mathcal{C}). \forall f : Hom(JI). J \sqsubset_R f \rightarrow |A| J) \rightarrow |A| I$

Definition 16 (Proof irrelevant sheaf). A proof irrelevant sheaf on $\mathcal{C}$ is a sheaf such that

$$\forall I. \forall x y : |A| I. x = y$$

Definition 17 (Category of elements). Let $\mathcal{C}$ be a category and F a functor from $\mathcal{C}$ to the category of sets, then its category of elements $\int F$ is given by the following:

- Its objects are pairs

$$(c, f) : Ob(\mathcal{C}) \times |F| c$$

- Its morphisms from (c, x) to (c', x') are morphisms $u : Hom_{\mathcal{C}}(c, c')$ such that

$$x \langle u \rangle_F = x'$$

Definition 18 (Dependent presheaf/sheaf). A dependent presheaf/sheaf A on $\mathcal{C}$ is the category of elements of A ($\int A$).

A Beth model is defined similarly to the Kripke model, we have as parameters a category $\mathcal{C}$ and a Grothendieck covering $\mathcal{J}$ and a presheaf D over $\mathcal{C}$. D is used to interpret relation and function symbols. The reason why D has to be a proper

presheaf instead of upper set, like in the propositional case, is because now we have term variables. In the propositional case we only had proof variables which had the property that for a given formula A its proofs are equal. For terms we don't have this so we need more structure.

Definition 19 (Beth semantics ($\mathcal{U}$)). Let $\mathcal{C}$ be a category, D a presheaf over $\mathcal{C}$ and $\mathcal{J}$ a Grothendieck covering on $\mathcal{C}$. Given the following components

$$\begin{aligned} \text{rel} &: \forall(n : \mathbb{N}). \text{relar } n \rightarrow (I : \text{Ob}(\mathcal{C})) \rightarrow \text{Vec}(|D| \ I) \ n \rightarrow \text{Prop} \\ \text{fun} &: \forall(n : \mathbb{N}). \text{relar } n \rightarrow (I : \text{Ob}(\mathcal{C})) \rightarrow \text{Vec}(|D| \ I) \ n \rightarrow |D| \ I \\ \langle \text{rel} \rangle &: \text{rel } n \ a \ I \ ds \rightarrow (I : \text{Hom}(J, I)) \rightarrow \text{rel } n \ a \ J \ (\text{map } ds \ (-\langle f \rangle_D)) \\ \langle \text{fun} \rangle &: (\text{fun } n \ a \ I \ ds) \langle f \rangle_D = \text{fun } n \ a \ J \ (\text{map } ds \ (-\langle f \rangle_D)) \end{aligned}$$

We define a model of the split Fol_{GAT}^I .

Term contexts are presheafs over $\mathcal{C}$. Term substitution are natural transformations between presheafs. Term substitution composition is given by composition of natural transformations and the identity term substitution is the identity natural transformation.

The empty term context $\diamond_t$ is the terminal presheaf. ϵ_t is the unique natural transformation into $\diamond_t$.

Formulas in context Γ are interpreted by proof irrelevant dependent sheafs on $\mathcal{C}$.

This can be seen as sheafs on the category of elements of $\int \Gamma$.

Terms in a context Γ are interpreted by natural transformations from Γ into D .

Term substitution is then composition of natural transformations, context extension is given by the product of Γ with D . Term vectors are interpreted in a similar way but indexed by a natural number, as natural transformations from Γ into $D' \ n$ where $D' \ n$ is

$$\begin{aligned} D' \ n &: \text{Con}_t \\ |D' \ 0| \ I &:= \mathbf{1} \\ |D' \ (n+1)| \ I &:= |D' \ n| \ I \times |D| \ I \\ \dots & \\ \text{Tms } \Gamma \ n &: \text{Set} \\ \text{Tms } \Gamma \ n &:= \text{Sub}_t \Gamma \ (D' \ n) \end{aligned}$$

Function symbols are interpreted in the following way.

$$\begin{aligned} \text{fun}' \ n \ a \ ts & : \text{Tm } \Gamma \\ |\text{fun}' \ n \ a \ ts| \ I \ x & := \text{fun } n \ a \ (\text{recTms } I \ (|ts| \ I \ x)) \\ \dots & \end{aligned}$$

where recTms has type $(I : \text{Ob}) \rightarrow |D' \ n| \ I \rightarrow \text{Vec } (D \ I) \ n$. The naturality is proven using $\langle \text{fun} \rangle$.

Relation symbols are interpreted by a sieve.

$$\begin{aligned} \text{rel-sieve } \Gamma \ n \ a \ ts \ I \ i & : \text{Sieve } I \\ J \sqcap_{\text{rel-sieve}} f & := \text{rel } n \ a \ J \ (\text{recTms } J \ (|ts| \ J \ (i \langle f \rangle_{\Gamma}))) \\ \dots & \\ \text{rel}' \ n \ a \ ts & : \text{For } \Gamma \\ |\text{rel}' \ n \ a \ ts| \ I \ i & := I \triangleleft \text{rel-sieve } \Gamma \ n \ a \ ts \ I \ i \\ x \langle f \rangle_{\text{rel}' \ n \ a \ ts} & := \text{substp } \text{eq}_1 \ (x [f]^c) \\ \text{glue}_{\text{rel}' \ n \ a \ ts} \ c \ x & := \text{local } c \ (f \ y \mapsto \text{substp } \text{eq}_2 \ (x \ f \ y)) \end{aligned}$$

and restr is given using $\langle \text{rel} \rangle$. We have to use substp at multiple places, this is because we need to transport along some equalities, here marked eq_n s, which are not interesting.

Proof contexts over a term context Γ are proof irrelevant dependent presheafs over Γ . Proof substitution are proof irrelevant natural transformations. Proofs of a formula K in context Γ are also proof irrelevant natural transformations from Γ to K .

The formulas are given by the following sheafs.

$\perp$.

$$\begin{aligned} \perp \text{sieve} & : \text{Sieve } I \\ I \sqcap_{\perp \text{sieve}} f & := \mathbf{0}_p \\ \text{restr}_{\perp \text{sieve}} \ b \ g & := \text{ind}_{\mathbf{0}_p} \ b \end{aligned}$$

$$\begin{aligned}
 \perp & : \text{For } \Gamma \\
 |\perp| \ I \ x & := I \triangleleft \perp \text{sieve} \\
 x \langle f \rangle_{\perp} & := x[f]^c \\
 \text{glue}_{\perp} \ c \ x & := \text{local } c \ x
 \end{aligned}$$

$\perp$ is given the empty sieve covering I . Gluing is then just locality.

$\top$.

$$\begin{aligned}
 \top & : \text{For } \Gamma \\
 |\top| \ I \ x & := \mathbf{1} \\
 x \langle f \rangle_{\top} & := * \\
 \text{glue}_{\top} \ c \ x & := *
 \end{aligned}$$

$\top$ is just the terminal presheaf, for which gluing is trivial.

$K \wedge L$.

$$\begin{aligned}
 K \wedge L & : \text{For } \Gamma \\
 |K \wedge L| \ I \ x & := |K| \ I \ x \times_p |L| \ I \ x \\
 (x, y) \langle f \rangle_{K \wedge L} & := (x \langle f \rangle_K, y \langle f \rangle_L) \\
 \text{glue}_{K \wedge L} \ c \ x & := (\text{glue}_K \ c \ (f \ y \mapsto \text{proj}_1 \ (x \ f \ y)), \text{glue}_L \ c \ (f \ y \mapsto \text{proj}_2 \ (x \ f \ y)))
 \end{aligned}$$

$\wedge$ is given by the product of two sheafs K and L . Gluing then follows from gluing on K and L .

$K \vee L$.

$$\begin{aligned}
 \vee \text{ sieve} & : (\Gamma : \text{Con}_t) \rightarrow (I : \text{Ob}(\mathcal{C})) \rightarrow \\
 & (x : |\Gamma|I) \rightarrow (K \ L : \text{For } \Gamma) \rightarrow \text{Sieve } I \\
 J \square_{\vee \text{ sieve}} f & := |K| \ J \ (x \langle f \rangle_{\Gamma}) +_p |L| \ J \ (x \langle f \rangle_{\Gamma}) \\
 \text{restr}_{\vee \text{ sieve}} (\text{inj}_1 \ K_j) \ g & := \text{substp eq}_1 (\text{inj}_1 \ (K_j \langle f \rangle_K)) \\
 \text{restr}_{\vee \text{ sieve}} (\text{inj}_2 \ L_j) \ g & := \text{substp eq}_2 (\text{inj}_2 \ (L_j \langle f \rangle_L)) \\
 K \vee L : \text{For } \Gamma & \\
 |K \vee L| \ I \ x & := I \triangleleft (\vee \text{ sieve } \Gamma \ I \ x \ K \ L) \\
 x \langle f \rangle_{K \vee L} & := \text{substp eq}_3 (x[f]^c) \\
 \text{glue}_{K \vee L} \ c \ x & := \text{local } c \ (f \ y \mapsto \text{substp eq}_4 \ (x \ f \ y))
 \end{aligned}$$

For disjunction we define the sieve $\vee\text{sieve}$ which uses $- +_p -$. Here we show how to prove eq_1 .

The expression $K_j \langle f \rangle_K$ has type $|K| K' (x \langle f \rangle_\Gamma) \langle g \rangle_\Gamma$ but the needed type is $|K| K' (x \langle f \circ g \rangle_\Gamma)$, this is proven with $\text{sym } \langle \circ \rangle_\Gamma$.

$K \supset L$.

$$\begin{aligned}
 K \supset L & : \text{For } \Gamma \\
 |K \supset L| \ Ix & := \forall(J : \text{Ob}(\mathcal{C})). \forall(f : \text{Hom}(J, I)). \\
 & |K| J (x \langle f \rangle_K) \rightarrow |L| J (x \langle f \rangle_L) \\
 x \langle f \rangle_{K \supset L} \ M \ g \ y & := \text{substp eq}_1 (x \ M \ (f \circ g) \ (\text{substp eq}_2 \ y)) \\
 \text{glue}_{K \supset L} \ c \ x \ J \ f \ K_j & := \text{glue}_L (c [f]^\circ) \\
 & (M \ g \ y \mapsto \text{substp eq}_3 (x \ (f \circ g) \ y \ M \ \text{id} \ (\text{substp eq}_4 (K_j \langle g \rangle_K))))
 \end{aligned}$$

Implication is given similarly to the Kripke semantics case and glue is given by gluing of L .

$\forall K$.

$$\begin{aligned}
 \forall K & : \text{For } \Gamma \\
 |\forall K| \ Ix & := \forall(J : \text{Ob}(\mathcal{C})). \forall(f : \text{Hom}(J, I)). \forall(d : D \ J). |K| J ((x \langle f \rangle_K), d) \\
 x \langle f \rangle_{\forall K} \ M \ g \ d & := \text{substp eq}_1 (x \ M \ (f \circ g) \ d) \\
 \text{glue}_{\forall K} \ c \ x \ J \ f \ d & := \text{glue}_K (c [f]^\circ) \\
 & (M \ g \ y \mapsto \text{substp eq}_2 (x \ (f \circ g) \ y \ M \ \text{id} \ (\text{substp eq}_3 (d \langle g \rangle_D))))
 \end{aligned}$$

Universal quantification is similar to implication but instead of $|K| J (x \langle f \rangle_K)$ we quantify over elements of D , $\forall(d : D \ J)$.

$\exists K$.

$$\begin{aligned}
 \exists\text{sieve} & : (\Gamma : \text{Con}_t) \rightarrow (I : \text{Ob}(\mathcal{C})) \rightarrow \\
 & (x : |\Gamma| \ I) \rightarrow (K \ L : \text{For } (\Gamma \triangleright_t)) \rightarrow \text{Sieve } I \\
 J \sqcap_{\exists\text{sieve}} f & := \Sigma_p (D \ J) (d \mapsto |K| J ((x \langle f \rangle_K), d)) \\
 \text{restr}_{\exists\text{sieve}} (D_j, K_j) \ g & := (D_j \langle g \rangle_D, \text{substp eq}_1 (K_j \langle g \rangle_K)) \\
 \exists K : \text{For } \Gamma & \\
 |\exists K| \ Ix & := I \triangleleft (\exists\text{sieve } \Gamma \ I \ x \ K) \\
 x \langle f \rangle_{\exists K} & := \text{substp eq}_2 (x [f]^\circ) \\
 \text{glue}_{\exists K} \ c \ x & := \text{local } c \ (f \ y \mapsto \text{substp eq}_3 (x \ f \ y))
 \end{aligned}$$

Existential quantification is given by a sieve which uses the meta level existential quantifier Σ_p . Restriction is component wise and glue is given by **local**.

Eq.

$$\begin{aligned} \text{Eqsieve} & : (\Gamma : \text{Cont}) \rightarrow (I : \text{Ob}(\mathcal{C})) \rightarrow (x : |\Gamma| I) \rightarrow (t \ t' : \text{Tm } \Gamma) \rightarrow \text{Sieve } I \\ J \square_{\text{Eqsieve}} f & := |t| J (x \langle f \rangle_\Gamma) = |t'| J (x \langle f \rangle_\Gamma) \\ \text{restr}_{\text{Eqsieve}} x \ g & := \text{eq}_1 \end{aligned}$$

$$\begin{aligned} \text{Eq } t \ t' : \text{For } \Gamma & \\ |\text{Eq } t \ t'| I \ x & := I \triangleleft (\text{Eqsieve } \Gamma \ I \ x \ t \ t') \\ x \langle f \rangle_{\text{Eq } t \ t'} & := \text{substp eq}_2 (x [f]^c) \\ \text{glue}_{\text{Eq } t \ t'} c \ x & := \text{local } c (f \ y \mapsto \text{substp eq}_3 (x \ f \ y)) \end{aligned}$$

The equality predicate uses the meta level equality, its restriction is a proof of the equation

$$(|t| K (I \langle f \rangle_\Gamma) = |t'| K (I \langle f \rangle_\Gamma)) \rightarrow |t| K (I \langle f \circ g \rangle_\Gamma) = |t'| K (I \langle f \circ g \rangle_\Gamma)$$

intro and elim The introduction and elimination rules for the connectives $\top, \wedge, \supset$ are the same as in the Kripke model. For the positive connectives namely $\perp, \vee, \exists$, **Eq** the introduction rules are given via **maximal** while the elimination rules use gluing. For example the elimination rule for $\perp$ is given by **glue** and the $\vee$ introductions are given by **maximal**.

$$\begin{aligned} \perp \text{elim} & : \text{Pf } \Gamma \ \perp \rightarrow \text{Pf } \Gamma \ A \\ |\perp \text{elim} x| \ c & := \text{glue}_K (|x| \ c) (f \ b \mapsto \text{ind} \mathbf{0}_p \ b) \\ \vee \text{intro}_1 : \text{Pf } \Gamma \ A \rightarrow \text{Pf } \Gamma \ (A \vee B) & \\ |\vee \text{intro}_1 x| \ c & := \text{maximal} (\text{inj}_1 ((|x| \ c) \langle \text{id} \rangle_\Gamma)) \end{aligned}$$

8.2 Soundness and completeness for Beth models

The proof of soundness is given via the iterator.

Soundness of Beth models for Fol_{GAT}^I ($\mathcal{U}$). Soundness is given by the Pf component of the iterator.

$$\vdash \neg^{\text{Pf}} : \text{Pf}_I \Gamma A \rightarrow \text{Pf} \vdash \Gamma \neg^C \vdash A \neg^F$$

□

For the proof of completeness we have to choose a category and a topology on it. For this we define sort of glued together contexts.

$$\begin{aligned} \text{Con} &:= \Sigma \text{Con}_t (\Gamma \mapsto \text{Con}_p \Gamma) \\ \text{Sub}(\Delta_t, \Delta_p)(\Gamma_t, \Gamma_p) &:= \Sigma (\text{Sub}_t \Delta_t \Gamma_t) (\gamma \mapsto \text{Sub}_p \Delta (\Gamma[\gamma]^C)) \\ \text{id} &:= (\text{id}_t, \text{substp eq}_1 \text{id}_p) \\ (\gamma_t, \gamma_p) \circ (\delta_t, \delta_p) &:= (\gamma_t \circ_t \delta_t, (\text{substp eq}_2 (\gamma_p [\delta_t]^S)) \circ_p \delta_p) \\ &\dots \end{aligned}$$

We refer to the term/proof component of Γ or γ with subscripts $\Gamma_t, \gamma_t, \Gamma_p$. Furthermore we have $\mathbf{p}$ and $\mathbf{q}$ for this glued notion of context as well.

$$\begin{aligned} \mathbf{q}'_t &: \text{Tm}(\Gamma_t \triangleright_t) \\ \mathbf{q}'_t &:= \mathbf{q}_{t_I} \\ \\ \mathbf{p}'_t &: \text{Sub}(\Gamma_t \triangleright_t, \Gamma_p) \Gamma \\ \mathbf{p}'_t &:= (\mathbf{p}_{t_I}, \text{id}_{p_I}) \\ \\ \mathbf{p}'_p &: \text{Sub}(\Gamma_t, \Gamma_p \triangleright_p K) \Gamma \\ \mathbf{p}'_p &:= \text{id}_{t_I}, (\text{substp eq}_1 \mathbf{p}_{p_I}) \\ \\ \mathbf{q}'_p &: \text{Pf}_I(\Gamma_p \triangleright_p K)(K[\text{id}_{t_I}]^F) \\ \mathbf{q}'_p &:= \text{substp eq}_2 \mathbf{q}_{p_I} \end{aligned}$$

We then have to define the topology on $\mathcal{C}$.

$$- \triangleleft - : (\Gamma : \text{Con}) \rightarrow (R : \text{Sieve } R) \rightarrow \text{Prop}$$

$$\begin{aligned} \text{maximal} &: \Gamma \square_R \text{id} \rightarrow \Gamma \triangleleft R \\ \triangleleft - \perp &: \text{Pf}_I \Gamma_t \perp \rightarrow \Gamma \triangleleft R \end{aligned}$$

$$\begin{aligned}
 \triangleleft - \vee \quad & : \\
 & (\forall(\gamma : \text{Sub } \Delta \Gamma). \text{Pf}_I \Delta_p (A[\gamma_t]^F) \rightarrow \Delta \triangleleft (R[\gamma]^S)) \rightarrow \\
 & (\forall(\gamma : \text{Sub } \Delta \Gamma). \text{Pf}_I \Delta_p (B[\gamma_t]^F) \rightarrow \Delta \triangleleft (R[\gamma]^S)) \rightarrow \\
 & \text{Pf}_I \Gamma_p (A \vee_I B) \rightarrow \Gamma \triangleleft R \\
 \triangleleft - \exists \quad & : (\forall(\gamma : \text{Sub } \Delta \Gamma). (d : \text{Tm}_I \Delta_t) \rightarrow \text{Pf}_I \Delta_p (A[\gamma_t, t]^F) \rightarrow \Delta \triangleleft (R[\gamma]^S)) \rightarrow \\
 & \text{Pf}_I \Gamma_p (\exists_I A) \rightarrow \\
 & \Gamma \triangleleft R \\
 \triangleleft - \text{Eq} \quad & : \text{Pf}_I \Gamma_p (\text{Eq}_I t t') \rightarrow \\
 & (\forall(\gamma : \text{Sub } \Delta \Gamma). \Delta \triangleleft (R[\gamma]^S)) \rightarrow \\
 & R = R' [\text{id}_t, t']^S \rightarrow \\
 & \Gamma \triangleleft R
 \end{aligned}$$

We have an constructor for **maximal** and for every eliminator of the positive formulas. The substitution operation and **local** is defined via induction on $\Gamma \triangleleft R$.

We choose D to be the presheaf of syntactic terms with term substitution, and **rel** and **fun** are the following

$$\begin{aligned}
 \text{rel } n \ a \ \Gamma \ ts &:= \text{Pf}_I \Gamma_p (\text{rel}_I n \ a \ (\text{reifyTms } ts)) \\
 \text{fun } n \ a \ \Gamma \ ts &:= \text{fun}_I n \ a \ (\text{reifyTms } ts)
 \end{aligned}$$

where **reifyTms** is the inverse of **recTms** turning vectors of **Tms** into **Tms**. We can define **reflect** and **reify** functions:

$$\begin{aligned}
 \text{reflect-cont} &: (\Delta_t : \text{Con}_{t_I}) \rightarrow (\gamma_t : \text{Sub}_{t_I} \Gamma_t \Delta_t) \rightarrow |\ulcorner \Delta_t \urcorner \text{Con}_t| \ \Gamma \\
 \text{reflect-conp} &: (\Delta : \text{Con}) \rightarrow (\gamma : \text{Sub } \Gamma \Delta) \rightarrow |\ulcorner \Delta_p \urcorner \text{Con}_p| \ \Gamma \ (\text{reflect-cont } \Delta_t \ \gamma_t) \\
 \text{reflect-Tm} &: |\ulcorner t \urcorner \text{Tm}| \ \Delta \ (\text{reflect-cont } \Gamma_t \ \gamma_t) = t [\gamma_t]_I^t \\
 \text{reify} &: (A : \text{For}_I \Gamma_t) \rightarrow |\ulcorner A \urcorner \text{For}| \ \Delta \ (\text{reflect-cont } \Gamma_t \ \gamma_t) \rightarrow \text{Pf}_I \Delta_p (A[\gamma_t]_I^F) \\
 \text{reflect} &: (A : \text{For}_I \Gamma_t) \rightarrow \text{Pf}_I \Delta_p (A[\gamma_t]_I^F) \rightarrow |\ulcorner A \urcorner \text{For}| \ \Delta \ (\text{reflect-cont } \Gamma_t \ \gamma_t)
 \end{aligned}$$

These functions are defined mutually inductively. The function **reflect-cont** is defined via induction on γ_t , and **reflect-conp** is defined via induction on Δ . It reflects the formulas in Δ by **reflect**

$$\text{reflect-conp } \diamond_p \ \gamma = *$$

$$\text{reflect-conp } (\Delta \triangleright_p K) \gamma = (\text{reflect-conp } \Delta (\gamma_t, (p_{p_I} \circ_{p_I} \gamma_p)), \text{reflect } K (q_{p_I} [\gamma]_I^P))$$

The main logic is happening in **reflect** and **reify**. We show **reflect** first. Its defined via induction on the formula A . The connectives $\top, \wedge, \supset$ are defined like in the propositional case, except lots of transport is needed like in the definition of the connectives in the Beth model. For the connectives which were defined by a sieve, we use the relevant part of $\Gamma \triangleleft A$.

$$\begin{aligned} \text{reflect } \perp x &= \triangleleft - \perp x \\ \text{reflect } (A \vee B) x &= \triangleleft - \vee \\ &\quad (\delta P_A \mapsto \text{maximal } (\text{inj}_1 (\text{substp eq}_1 (\text{reflect } A (\text{substp eq}_2 P_A)))))) \\ &\quad (\delta P_B \mapsto \text{maximal } (\text{inj}_2 (\text{substp eq}_3 (\text{reflect } B (\text{substp eq}_4 P_B)))))) \\ &\quad x \\ \text{reflect } (\exists A) x &= \triangleleft - \exists \\ &\quad (\delta d P_A \mapsto \text{maximal } (d, (\text{substp eq}_5 (\text{reflect } A (\text{substp eq}_6 P_A)))))) \\ &\quad x \\ &\quad \dots \end{aligned}$$

reify is more complicated, for the connectives $\top, \wedge, \supset$ it is, like **reflect**, defined similarly like in the propositional case. For the positive connectives and relations we define it via specialisation to the connectives so we get the functions.

$$\begin{aligned} \text{reify-}\perp &: |\ulcorner \perp_I \urcorner^{\text{For}}| \Delta (\text{reflect-cont } \Gamma_t \gamma_t) \rightarrow \text{Pf}_I \Delta_p \perp_I \\ \text{reify-}\vee &: (A B : \text{For}_I \Gamma_t) \rightarrow |\ulcorner A \vee_I B \urcorner^{\text{For}}| \Delta (\text{reflect-cont } \Gamma_t \gamma_t) \rightarrow \text{Pf}_I \Delta_p ((A \vee_I B) [\gamma_t]_I^F) \\ \text{reify-}\exists &: (A : \text{For}_I (\Gamma_t \triangleright_{t_I})) \rightarrow |\ulcorner \exists_I A \urcorner^{\text{For}}| \Delta (\text{reflect-cont } \Gamma_t \gamma_t) \rightarrow \text{Pf}_I \Delta_p ((\exists_I A) [\gamma_t]_I^F) \\ \text{reify-Eq} &: (t t' : \text{Tm}_I \Gamma_t) \rightarrow |\ulcorner \text{Eq}_I t t' \urcorner^{\text{For}}| \Delta (\text{reflect-cont } \Gamma_t \gamma_t) \rightarrow \text{Pf}_I \Delta_p ((\text{Eq}_I t t') [\gamma_t]_I^F) \\ \text{reify-rel} &: \dots \end{aligned}$$

These functions are then given by induction on $-\triangleleft-$, the second argument of **reflect**. We show the case for disjunction.

$$\begin{aligned} \text{reify-}\vee A B (\text{maximal } (\text{inj}_1 x)) &:= \vee\text{intro}_{1_I} (\text{reify } A (\text{substp eq}_1 x)) \\ \text{reify-}\vee A B (\text{maximal } (\text{inj}_2 x)) &:= \vee\text{intro}_{2_I} (\text{reify } B (\text{substp eq}_2 x)) \\ \text{reify-}\vee A B (\triangleleft - \perp x) &:= \perp\text{elim}_I x \\ \text{reify-}\vee A B (\triangleleft - \vee f g x) &:= \end{aligned}$$

$$\begin{aligned}
 & \vee \text{elim}_I \\
 & (\text{reify-}\vee A B (\text{substp eq}_3 (f \text{ p}'_p \text{ q}'_p))) \\
 & (\text{reify-}\vee A B (\text{substp eq}_4 (g \text{ p}'_p \text{ q}'_p))) \\
 & x \\
 \text{reify-}\vee A B (\triangleleft - \exists f x) & := \\
 & \exists \text{elim}_I x \\
 & (\text{substp eq}_5 (\text{reify-}\vee A B (\text{substp eq}_6 (f (\text{p}_{t_I}, \text{p}_{p_I}) \text{ q}'_t \text{ q}_{p_I})))) \\
 \text{reify-}\vee A B (\triangleleft - \text{Eq } x f p) & := \text{reify-}\vee A B (\text{substp eq}_7 (f \text{ id}))
 \end{aligned}$$

For the $\triangleleft - \vee$ case is given by the introduction rules, and the others are by their elimination rules. This is the format of the others reify functions also. Then completeness is composing the reflecting and reifying functions.

Completeness of Fol_{GAT}^I ($\mathcal{U}$).

$$\text{completeness} : \forall A \rightarrow \text{Pf} \vdash \Gamma \neg^C \vdash A \neg^F \rightarrow \text{Pf}_I \Gamma A$$

is given by the term

$$\Gamma A p \mapsto \text{substp eq}_1 (\text{reify } A (|p| (\text{reflect-conp } \Gamma_p \text{ id}_{t_I} (\text{substp eq}_2 \text{ id}_{p_I}))))$$

□

Completeness is the composition of **reify** and **reflect**. Given Γ we first reflect its proof part via **reflect-conp**, while not substituting. p is the semantic proof of A in context Γ_p , in the semantics elements of **Pf** are functions from $|\Gamma \neg^C| I i$ to $|\Gamma A \neg^F| I i$. Given the reflected context to p we get $|\Gamma A \neg^F| \Gamma (\text{reflect-cont id}_{t_I})$ which we can then reify back into a syntactic proof of A .

Chapter 9

Double-negation translation

The first double-negation translation was given by Kolmogorov [21] during the debate between Hilbert and Brouwer [22, 23] about the foundations of mathematics. His motivation was to give a translation from classical logic to intuitionistic logic such that a statements is provable in classical logic *if and only if* its image is provable under this translation. Kolmogorov's idea was to notice that the double negation of classical statements hold intuitionistically. Since then, there have been multiple double negation translations [24]. In our presentation the double negation translation is a model construction, it constructs a model of first-order classical logic $\mathbf{Fol}^C$ from the syntax of first-order intuitionistic logic $\mathbf{Fol}^I$. This is done by interpreting formulas as their double negated versions, however we use a minimal version which only double negates the positive connectives $\vee, \exists, \mathbf{Eq}$. The difference between the two logics, $\mathbf{Fol}^C$ and $\mathbf{Fol}^I$ is the axiom of the *Law of excluded middle* (LEM). This axiom states :

$$P \vee \neg P$$

that is, for any proposition P one can either prove P or its negation $\neg P$. This is equivalent to the axiom of *Double negation elimination*, that is they mutually imply each other [25].

$$\neg\neg P \supset P$$

The axiom of *Double negation elimination* states that for all propositions P we have a way to prove it by assuming its false, and then arriving at a contradiction. Intuitionistically or more broadly constructively *Law of excluded middle* and *Double negation elimination* is rejected [ref]. When constructing the model of classical logic

we have to ensure that we can also prove the *Law of excluded middle*, this is done by formulas carrying with them a proof of their *Double negation elimination*. We give the translation component wise.

Definition 20 (Double negation translation ($\llbracket \cdot \rrbracket$)). Contexts and substitutions are given by contexts and substitutions of the syntax. Formulas are given by 2 components:

- A syntactic formula A in the given context Γ
- A proof of the double negation of A

$$\text{Pf } \Gamma (\neg\neg A \supset A)$$

Formula substitutions is then given components wise, the first components is substituted as a formula and the second is with proof substitution $-[-]^P$. Proofs and proof substitutions are just their syntactic counterparts, the same is true for Terms and term telescopes (**Tms**). The formula component of the connectives are interpreted in the following way :

$$\begin{aligned} \top &:= \top \\ \perp &:= \perp \\ A \wedge B &:= A \wedge B \\ A \vee B &:= \neg\neg(A \vee B) \\ A \supset B &:= A \supset B \\ \forall A &:= \forall A \\ \exists A &:= \neg\neg(\exists A) \\ \text{Eq } t t' &:= \neg\neg(\text{Eq } t t') \end{aligned}$$

Only the positive, $\vee$, $\exists$, **Eq**, connectives are double negated. We show the other component of the formulas, these are the proofs of double negation.

$\perp$. We need to prove

$$\neg\neg\perp \supset \perp$$

because of the definition of $\neg-$ as $- \supset \perp$ we have

$$((\perp \supset \perp) \supset \perp) \supset \perp$$

$$\frac{\frac{\Gamma, ((\perp \supset \perp) \supset \perp) \vdash ((\perp \supset \perp) \supset \perp)}{\Gamma, ((\perp \supset \perp) \supset \perp) \vdash \perp \supset \perp} \supset \text{intro} \quad \frac{\Gamma, ((\perp \supset \perp) \supset \perp), \perp \vdash \perp}{\Gamma, ((\perp \supset \perp) \supset \perp) \vdash \perp \supset \perp} \$}{\Gamma \vdash ((\perp \supset \perp) \supset \perp) \supset \perp} \supset \text{intro}$$

$\top$. For $\top$ the double negation proof is

$$\frac{\frac{\Gamma, ((\top \supset \perp) \supset \perp) \vdash \top}{\Gamma \vdash ((\top \supset \perp) \supset \perp) \supset \top} tt}{\Gamma \vdash ((\top \supset \perp) \supset \perp) \supset \top} \supset \text{intro}$$

In the following cases we use the fact that formulas have their double negation proofs with them, we will denote them with P_a, P_b .

$$P_a : \neg\neg A \supset A$$

$$P_b : \neg\neg B \supset B$$

$A \supset B$.

$$\begin{array}{c}
 (1) \\
 P_b \frac{\frac{\overline{\Gamma, \neg\neg(A \supset B), A \vdash \neg\neg B \supset B} \quad \overline{\Gamma, \neg\neg(A \supset B), A \vdash \neg\neg B}}{\Gamma, \neg\neg(A \supset B), A \vdash B} \$}{\Gamma, \neg\neg(A \supset B) \vdash A \supset B} \supset \text{intro} \\
 \hline
 \Gamma \vdash \neg\neg(A \supset B) \supset (A \supset B) \quad \supset \text{intro}
 \end{array}$$

$$\begin{array}{c}
 (2) \\
 \frac{\frac{\overline{\Gamma, \neg\neg(A \supset B), A, \neg B, A \supset B \vdash \perp}}{\Gamma, \neg\neg(A \supset B), A, \neg B \vdash \neg(A \supset B)} \supset \text{intro} \quad \overline{\Gamma, \neg\neg(A \supset B), \dots \vdash \neg\neg(A \supset B)}}{\Gamma, \neg\neg(A \supset B), A, \neg B \vdash \perp} \$ \\
 \hline
 \Gamma, \neg\neg(A \supset B), A \vdash \neg\neg B \quad \supset \text{intro} \\
 \hline
 (1)
 \end{array}$$

$$\begin{array}{c}
 \frac{\overline{\Gamma, \dots, \neg B, A \supset B \vdash \neg B} \quad \frac{\overline{\Gamma, \dots, A \supset B \vdash A \supset B} \quad \overline{\Gamma, \dots, A, \dots \vdash A}}{\Gamma, \neg\neg(A \supset B), A, \neg B, A \supset B \vdash B} \$}{\Gamma, \neg\neg(A \supset B), A, \neg B, A \supset B \vdash \perp} \$ \\
 \hline
 (2)
 \end{array}$$

$\wedge$.

$$\begin{array}{c}
 \begin{array}{c}
 (1) \\
 \frac{\frac{\Gamma, \neg\neg(A \wedge B), \neg A \vdash \perp}{\Gamma, \neg\neg(A \wedge B) \vdash \neg\neg A}}{P_a \frac{\Gamma, \neg\neg(A \wedge B) \vdash A}{\Gamma, \neg\neg(A \wedge B) \vdash A \wedge B}} \\
 \frac{\Gamma, \neg\neg(A \wedge B) \vdash A \wedge B}{\Gamma \vdash \neg\neg(A \wedge B) \supset A \wedge B} \supset \text{intro}
 \end{array}
 \quad
 \begin{array}{c}
 (2) \\
 \frac{\frac{\Gamma, \neg\neg(A \wedge B), \neg B \vdash \perp}{\Gamma, \neg\neg(A \wedge B) \vdash \neg\neg B}}{P_b \frac{\Gamma, \neg\neg(A \wedge B) \vdash B}{\Gamma, \neg\neg(A \wedge B) \vdash A \wedge B}} \\
 \frac{\Gamma, \neg\neg(A \wedge B) \vdash A \wedge B}{\Gamma \vdash \neg\neg(A \wedge B) \supset A \wedge B} \supset \text{intro}
 \end{array}
 \end{array}$$

$$\begin{array}{c}
 \frac{\frac{\Gamma, \dots, \neg A, \dots \vdash \neg A}{\Gamma, \dots, \neg A, A \wedge B \vdash A} \quad \frac{\Gamma, \dots, A \wedge B \vdash A \wedge B}{\Gamma, \dots, \neg A, A \wedge B \vdash A} \wedge \text{elim}_1}{\Gamma, \dots, \neg A, A \wedge B \vdash \perp} \$ \\
 \frac{\Gamma, \neg\neg(A \wedge B), \neg A \vdash \neg(A \wedge B)}{\Gamma, \neg\neg(A \wedge B), \neg A \vdash \perp} \supset \text{intro} \\
 \vdots \\
 \frac{\Gamma, \neg\neg(A \wedge B), \neg A \vdash \neg\neg(A \wedge B)}{\Gamma, \neg\neg(A \wedge B), \neg A \vdash \perp} \$ \\
 (1)
 \end{array}$$

$$\begin{array}{c}
 \frac{\frac{\Gamma, \dots, \neg B, \dots \vdash \neg B}{\Gamma, \dots, \neg B, A \wedge B \vdash B} \quad \frac{\Gamma, \dots, A \wedge B \vdash A \wedge B}{\Gamma, \dots, \neg B, A \wedge B \vdash B} \wedge \text{elim}_2}{\Gamma, \dots, \neg B, A \wedge B \vdash \perp} \$ \\
 \frac{\Gamma, \neg\neg(A \wedge B), \neg B \vdash \neg(A \wedge B)}{\Gamma, \neg\neg(A \wedge B), \neg B \vdash \perp} \supset \text{intro} \\
 \vdots \\
 \frac{\Gamma, \neg\neg(A \wedge B), \neg B \vdash \neg\neg(A \wedge B)}{\Gamma, \neg\neg(A \wedge B), \neg B \vdash \perp} \$ \\
 (2)
 \end{array}$$

$\forall$.

$$\begin{array}{c}
 (1) \\
 \frac{\frac{\Gamma, \neg\neg(\forall A), t \vdash \neg\neg A \supset A}{\Gamma, \neg\neg(\forall A), t \vdash A} \quad \frac{\Gamma, \neg\neg(\forall A), t, \neg A \vdash \perp}{\Gamma, \neg\neg(\forall A), t \vdash \neg\neg A} \supset \text{intro}}{P_a \frac{\Gamma, \neg\neg(\forall A), t \vdash A}{\Gamma, \neg\neg(\forall A) \vdash \forall A}} \$ \\
 \frac{\Gamma, \neg\neg(\forall A) \vdash \forall A}{\Gamma \vdash \neg\neg(\forall A) \supset \forall A} \supset \text{intro}
 \end{array}$$

$$\begin{array}{c}
 \frac{\frac{\Gamma, \dots, \neg A, \forall A \vdash \neg A}{\Gamma, \neg\neg(\forall A), t, \neg A, \forall A \vdash \perp} \quad \frac{\frac{\Gamma, \dots, \forall A \vdash \forall A \quad \Gamma, \dots, t, \dots \vdash t}{\Gamma, \neg\neg(\forall A), t, \neg A, \forall A \vdash A} \text{Velim}}{\Gamma, \neg\neg(\forall A), t, \neg A, \forall A \vdash \perp} \$ \\
 \frac{\Gamma, \neg\neg(\forall A), t, \neg A \vdash \neg\neg(\forall A)}{\Gamma, \neg\neg(\forall A), t, \neg A \vdash \neg(\forall A)} \supset \text{intro} \\
 \frac{\Gamma, \neg\neg(\forall A), t, \neg A \vdash \neg(\forall A)}{\Gamma, \neg\neg(\forall A), t, \neg A \vdash \perp} \$ \\
 \hline
 (1)
 \end{array}$$

$\forall, \exists, \mathbf{Eq}$. In these cases our we have to prove something of the following form :

$$\neg\neg(\neg\neg A) \supset (\neg\neg A)$$

This is true in general so we can apply it to get the proofs for $\forall, \exists, \mathbf{Eq}$.

$$\begin{array}{c}
 \frac{\frac{\Gamma, \dots, \neg\neg A \vdash \neg\neg A \quad \Gamma, \dots, \neg A, \dots \vdash \neg A}{\Gamma, \dots, \neg A, (\neg\neg A) \vdash \perp} \$}{\Gamma, \neg\neg(\neg\neg A), \neg A \vdash \neg\neg(\neg\neg A)} \supset \text{intro} \\
 \frac{\Gamma, \neg\neg(\neg\neg A), \neg A \vdash \neg\neg(\neg\neg A)}{\Gamma, \neg\neg(\neg\neg A), \neg A \vdash \perp} \$ \\
 \frac{\Gamma, \neg\neg(\neg\neg A), \neg A \vdash \perp}{\Gamma, \neg\neg(\neg\neg A) \vdash (\neg\neg A)} \supset \text{intro} \\
 \frac{\Gamma, \neg\neg(\neg\neg A) \vdash (\neg\neg A)}{\Gamma \vdash \neg\neg(\neg\neg A) \supset (\neg\neg A)} \supset \text{intro}
 \end{array}$$

LEM. Finally we can prove LEM from

$$\neg\neg A \supset A$$

$$\begin{array}{c}
 \frac{\frac{\Gamma, \neg(A \vee \neg A), A \vdash \neg(A \vee \neg A) \quad \Gamma, \neg(A \vee \neg A), A \vdash A \vee \neg A}{\Gamma, \neg(A \vee \neg A), A \vdash \perp} \text{Vintro}_1}{\Gamma, \neg(A \vee \neg A), A \vdash \neg(A \vee \neg A)} \$ \\
 \frac{\Gamma, \neg(A \vee \neg A), A \vdash \neg(A \vee \neg A)}{\Gamma, \neg(A \vee \neg A) \vdash \neg A} \supset \text{intro} \\
 \frac{\Gamma, \neg(A \vee \neg A) \vdash \neg A}{\Gamma, \neg(A \vee \neg A) \vdash A \vee \neg A} \text{Vintro}_2 \\
 \frac{\Gamma, \neg(A \vee \neg A) \vdash A \vee \neg A}{\Gamma, \neg(A \vee \neg A) \vdash \perp} \$ \\
 \frac{\Gamma, \neg(A \vee \neg A) \vdash \perp}{\Gamma \vdash \neg\neg(A \vee \neg A)} \supset \text{intro}
 \end{array}$$

Chapter 10

Further research

Our Beth/sheaf semantics can be naturally extended to give models of type theory with strong sums, that is definitional beta and eta rules for sum types [26]. We think our formalisation can prove helpfull for a formalisation of this result which has not been done yet. In chapter 5 we give a conjecture about the split and non-split notion of models. If this is proven, it could prove useful in defining a generic way to split contexts in GATs obtained through the SOGAT to GAT translation.

Chapter 11

Conclusion

In conclusion, our works demonstrates how we can describe first order logic as a second order generalised algebraic theory. We then use this description to derive Kripke and Beth models for it. We show how to prove completeness of Kripke models the negative fragment of propositional logic, and completeness of Beth models for full intuitionistic first order logic in a normalisation by evaluation fashion. Our formalisation also includes more notions of models such as the Bool model, family model, and also includes first order Kripke models and propositional Beth models, for which we also prove completeness for intuitionistic propositional logic. Finally we show the double negation translation from classical logic to intuitionistic logic in our framework.

Acknowledgements

I would like to thank my advisor Szumi Xie for his helpful comments on the thesis and his availability. I would also like to thank Kaposi Ambrus for suggesting the topic of my thesis and Kanalas Kristof for discussion on Categorical logic. I thank my parents and family for their continued help and support throughout my studies.

Appendix A

Appendix

In this appendix we give a few definition of category theory and some notation of our metatheory.

A.1 Metatheory

Our work is formalised in the Agda proof assistant so our metatheory is Per Martin-Löf type theory. We show some definitions that we use throughout the thesis.

Definition 21 (Pi type). Π is the dependent function type with, we just write $- \rightarrow -$, for example

$$f : (a : A) \rightarrow B \ a$$

We use

$$f := a \mapsto b$$

to mean an element of Π and we use $f \ a$ for function application.

Definition 22 (Sigma type). Σ is the dependent product type with constructor $- , -$ and destructors **proj**₁ and **proj**₂. The propositionally truncated version is written Σ_p .

Definition 23 (Product and sum type). We use $- \times -$ and $- + -$ for the product and sum types. Product introduction rule $- , -$ and destructors **proj**₁, **proj**₂. For the propositionally truncated product we write $- \times_p -$.

Sum constructors **inj**₁, **inj**₂ and **ind**₊ for eliminator. $- +_p -$ is the propositionally truncated version.

Definition 24 (Empty and unit type). **1** is the unit type with one element $*$. **0** is the empty type with eliminator **ind****0**_p.

Definition 25 (Vector). $\text{Vec } A n$ is the type of vectors with elements in A and length n . The constructors are $\text{nil} : \text{Vec } A 0$ and $\text{cons} : \text{Vec } A n \rightarrow \text{Vec } A (n + 1)$ and there is an operation $\text{map} : (A \rightarrow B) \rightarrow \text{Vec } A n \rightarrow \text{Vec } B n$

A.2 Category theory

We give a few definitions of category theory that we use in the thesis, we don't deal about size issues.

Definition 26 (Category). A category $\mathcal{C}$ has the following components.

- $Ob : Set$ - A set of objects,
- $Hom : Ob \rightarrow Ob \rightarrow Set$ - A set of arrows/morphisms between two objects.
We sometimes write $Hom(I, J)$.
- $\text{id} : Hom\ I\ I$ - Identity arrow/morphism for every object. We sometimes use the notation id_I for identity of the object I .
- $- \circ - : Hom\ J\ K \rightarrow Hom\ I\ J \rightarrow Hom\ I\ K$ - Composition of arrows.
- $\text{id} \circ f = f$
- $f \circ \text{id} = f$
- $(f \circ g) \circ h = f \circ (g \circ h)$

Definition 27 (Functor). A functor F between two categories $\mathcal{C}$ and $\mathcal{D}$ has the following components.

- $F_0 : Ob(\mathcal{C}) \rightarrow Ob(\mathcal{D})$ - An action on objects.
- $F_1 : Hom_{\mathcal{C}}(I, J) \rightarrow Hom_{\mathcal{D}}(F_0\ I, F_0\ J)$ - An action on morphisms.
- $F_1\ \text{id}_I = \text{id}_{F_0\ I}$
- $F_1\ (f \circ_{\mathcal{C}} g) = (F_1\ f) \circ_{\mathcal{D}} (F_1\ g)$

Definition 28 (Natural transformations). A natural transformations α between two functors F, G between categories $\mathcal{C}$ and $\mathcal{D}$ has the following components.

- $\alpha_0 : ?$
- $?$

Bibliography

- [1] van Dalen Dirk. “Logic and Structure”. In: Springer, 1980. Chap. Intuitionistic Logic.
- [2] Marcelo Fiore, Gordon Plotkin, and Daniele Turi. “Abstract Syntax and Variable Binding”. In: *Proceedings of the 14th Annual IEEE Symposium on Logic in Computer Science*. LICS '99. USA: IEEE Computer Society, 1999, p. 193. ISBN: 0769501583.
- [3] Taichi Uemura. “Abstract and concrete type theories”. PhD thesis. Institute for Logic, Language and Computation, University of Amsterdam, July 2021.
- [4] Runming Li, Yue Yao, and Robert Harper. “Mechanizing Synthetic Tait Computability in Istari”. In: *Proceedings of the 15th ACM SIGPLAN International Conference on Certified Programs and Proofs*. CPP '26. Rennes, France: Association for Computing Machinery, 2026, pp. 231–247. ISBN: 9798400723414. DOI: 10.1145/3779031.3779085. URL: <https://doi.org/10.1145/3779031.3779085>.
- [5] Thiago Felicissimo et al. “Definitional Proof Irrelevance Made Accessible”. working paper or preprint. 2026. URL: <https://hal.science/hal-05474391>.
- [6] Ambrus Kaposi and Szumi Xie. “Second-Order Generalised Algebraic Theories: Signatures and First-Order Semantics”. In: *9th International Conference on Formal Structures for Computation and Deduction (FSCD 2024)*. Ed. by Jakob Rehof. Vol. 299. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, 10:1–10:24. ISBN: 978-3-95977-323-2. DOI: 10.4230/LIPIcs.FSCD.2024.10. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2024.10>.

- [7] Julia Morin. *Joyals representation theorem for Heyting categories*. Accessed: 2026-02-14. 2024. URL: <https://web.archive.org/web/20260214203533/https://www.cmu.edu/dietrich/philosophy/hott/slides/masterthesis-juliamorin.pdf>.
- [8] Michael Makkai and Gonzalo E. Reyes. *First Order Categorical Logic*. Springer, 1977. DOI: 10.1007/BFb0066201.
- [9] Asta Halkjær From. “A Succinct Formalization of the Completeness of First-Order Logic”. In: *27th International Conference on Types for Proofs and Programs (TYPES 2021)*. Ed. by Henning Basold, Jesper Cockx, and Silvia Ghilezan. Vol. 239. Leibniz International Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 8:1–8:24. ISBN: 978-3-95977-254-9. DOI: 10.4230/LIPIcs.TYPES.2021.8. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TYPES.2021.8>.
- [10] *Formalized Formal Logic*. URL: <https://github.com/FormalizedFormalLogic>.
- [11] John Cartmell. “Generalised algebraic theories and contextual categories”. In: *Annals of Pure and Applied Logic* 32 (1986), pp. 209–243. ISSN: 0168-0072. DOI: [https://doi.org/10.1016/0168-0072\(86\)90053-9](https://doi.org/10.1016/0168-0072(86)90053-9). URL: <https://www.sciencedirect.com/science/article/pii/0168007286900539>.
- [12] Andras Kovács. “Type-Theoretic Signatures for Algebraic Theories and Inductive Types”. PhD thesis. Eötvös Loránd University, 2022.
- [13] Alonzo Church. “A Set of Postulates for the Foundation of Logic”. In: *Annals of Mathematics* 33.2 (1932), pp. 346–366. ISSN: 0003486X, 19398980. URL: <http://www.jstor.org/stable/1968337> (visited on 04/21/2026).
- [14] M. Hofmann. “Semantical analysis of higher-order abstract syntax”. In: *Proceedings. 14th Symposium on Logic in Computer Science (Cat. No. PR00158)*. 1999, pp. 204–213. DOI: 10.1109/LICS.1999.782616.
- [15] Rafaël Bocquet, Ambrus Kaposi, and Christian Sattler. “For the Metatheory of Type Theory, Internal Scoring Is Enough”. In: *8th International Conference on Formal Structures for Computation and Deduction (FSCD 2023)*. Ed. by Marco Gaboardi and Femke van Raamsdonk. Vol. 260. Leibniz International

- Proceedings in Informatics (LIPIcs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023, 18:1–18:23. ISBN: 978-3-95977-277-8. DOI: 10.4230/LIPIcs.FSCD.2023.18. URL: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.FSCD.2023.18>.
- [16] Fredrik Nordvall Forsberg. “Inductive-inductive definitions”. PhD thesis. Swansea University, 2013.
- [17] Thierry Coquand. “Canonicity and normalization for dependent type theory”. In: *Theoretical Computer Science* 777 (2019). In memory of Maurice Nivat, a founding father of Theoretical Computer Science - Part I, pp. 184–191. ISSN: 0304-3975. DOI: <https://doi.org/10.1016/j.tcs.2019.01.015>. URL: <https://www.sciencedirect.com/science/article/pii/S0304397519300325>.
- [18] Thorsten Altenkirch and Ambrus Kaposi. “Normalisation by Evaluation for Type Theory, in Type Theory”. In: *CoRR* abs/1612.02462 (2016). arXiv: 1612.02462. URL: <http://arxiv.org/abs/1612.02462>.
- [19] Callan McGill. *Kripke And Beth Semantics In Lean*. URL: <https://web.archive.org/web/20260428113435/https://boarders.github.io/posts/beth.html>.
- [20] Pierre-Marie Pédro. *Debunking Sheaves*. Accessed: 2026-02-16. URL: <https://web.archive.org/web/20260216210442/https://www.p%C3%A9dro.fr/drafts/sheafstt.pdf>.
- [21] Thierry Coquand. “Kolmogorovs contribution to intuitionistic logic”. In: Sept. 2007, pp. 19–40. ISBN: 978-3-540-36349-1. DOI: 10.1007/978-3-540-36351-4_2.
- [22] D. van Dalen. “The War of the Frogs and the Mice, or the Crisis of the Mathematische Annalen”. In: *Mathematical Conversations: Selections from The Mathematical Intelligencer*. New York, NY: Springer New York, 2001, pp. 445–465. ISBN: 978-1-4613-0195-0. DOI: 10.1007/978-1-4613-0195-0_40. URL: https://doi.org/10.1007/978-1-4613-0195-0_40.
- [23] S.C. Kleene and M. Beeson. *Introduction to Metamathematics*. Bibliotheca mathematica. Ishi Press International, 2009. ISBN: 9780923891572. URL: <https://books.google.hu/books?id=HZAjPwAACAAJ>.

- [24] Gilda Ferreira and Paulo Oliva. “On Various Negative Translations”. In: *Electronic Proceedings in Theoretical Computer Science* 47 (Jan. 2011), pp. 21–33. DOI: 10.4204/EPTCS.47.4.
- [25] Richard Moot and Christian Retoré. *Classical logic and intuitionistic logic: equivalent formulations in natural deduction, Gödel-Kolmogorov-Glivenko translation*. Research Report. LIRMM ; LABRI, Feb. 2016. URL: <https://hal-lirmm.ccsd.cnrs.fr/lirmm-01281243>.
- [26] Thorsten Altenkirch et al. “Normalization by evaluation for typed lambda calculus with coproducts”. In: *16th Annual IEEE Symposium on Logic in Computer Science*. 2001, pp. 303–310.